

Documentação Oficial do Sistema

Versão 1.0 • Novembro de 2025

Sumário

SUMÁRIO 0

1. INTRODUÇÃO 1

1.1. OBJETIVO 1
1.2 JUSTIFICATIVA 1

2. ANÁLISE DO SISTEMA PROPOSTO 2

2.1 ESCOPO DO SISTEMA 2
2.2 NÃO ESCOPO DO SISTEMA 2
2.3 REQUISITOS DO SISTEMA 3
2.4 GLOSSÁRIO 4
2.5 MAPEAMENTO DE PROCESSOS 4
2.6 DIAGRAMA DE CASO DE USO 6
2.7 CASO DE USO EXPANDIDO 8
2.8 DIAGRAMA DE CLASSE 10
2.9 TESTES DO SOFTWARE/APLICATIVO/SOLUÇÃO 11

3. PROJETO DE INTERFACE GRÁFICA (MODELO NÃO FUNCIONAL) 12

3.1 MODELO NÃO FUNCIONAL 12
3.2 INTERNACIONALIZAÇÃO DO SOFTWARE 22

4. PROJETO DE BANCO DE DADOS 24

4.1 DIAGRAMA DE ENTIDADE E RELACIONAMENTO (DER) 24
4.2 DIAGRAMA DE BANCO DE DADOS 26
4.3 QUADRO RESUMO DOS OBJETOS DE BANCO DE DADOS 26
4.4 MAPEAMENTO PARA O MODELO LÓGICO (MODELO RELACIONAL) 27
4.5 SCRIPT DE BANCO DE DADOS 27
4.6 INSERÇÕES NO BANCO DE DADOS 35
4.7 VIEWS 39
4.8 FUNÇÕES ESCALARES 39
4.9 FUNÇÕES DO TIPO TABLE 39
4.10 PROCEDURES 39
4.11 TRIGGERS 39

5. EVOLUÇÃO DO SISTEMA 40

5.1 PRÓXIMAS EVOLUÇÕES 40

REFERÊNCIAS 40

1. Introdução

Sistema: LumiFlow – Sistema Web de Gestão e Rastreabilidade da Produção Industrial

O LumiFlow é um sistema web criado para organizar, rastrear e automatizar o controle de produção de empresas industriais e de manufatura. Ele centraliza em uma única plataforma o planejamento, a execução e o acompanhamento das ordens de produção, substituindo controles manuais por informações confiáveis e atualizadas em tempo real.

Cada produto recebe um roteiro de fabricação que define a sequência de setores e etapas por onde ele passa. As ordens de produção percorrem esse roteiro automaticamente: a cada lançamento feito no chão de fábrica, o sistema registra quem produziu, em qual máquina e em que quantidade, e encaminha as peças para a etapa seguinte. Refugos, retrabalhos e movimentações de estoque ficam registrados, o que garante a rastreabilidade completa de cada ordem.

Este documento descreve o escopo, os requisitos, a modelagem, a interface, o banco de dados e a estratégia de testes do LumiFlow, e serve como referência técnica e funcional para usuários, implantadores e desenvolvedores.

1.1. Objetivo

Oferecer a empresas industriais uma plataforma web que organize, rastreie e automatize o controle de produção, reduzindo controles manuais e dando visibilidade em tempo real sobre o andamento das ordens de produção em cada setor do chão de fábrica.

Objetivos Específicos

- Modelar o fluxo produtivo em roteiros de fabricação compostos por setores e etapas;
- Permitir a abertura, a liberação, a divisão e o acompanhamento das ordens de produção;
- Registrar a produção, os refugos e os retrabalhos com rastreabilidade por operador e máquina;
- Controlar o estoque e a movimentação de insumos, como as chapas de vidro;
- Disponibilizar indicadores e histórico para apoiar a tomada de decisão da gestão;
- Garantir segurança de acesso por perfil e a integridade dos dados registrados.

1.2 Justificativa

Em muitas indústrias, o acompanhamento da produção ainda depende de controles manuais, planilhas e comunicação verbal entre os setores. Esse modelo dificulta saber em que etapa cada ordem se encontra, quanto já foi produzido e quanto ainda está pendente, além de tornar frágil o registro de refugos, retrabalhos e consumo de insumos.

Um sistema que acompanhe cada ordem ao longo do seu roteiro de fabricação permite que a gestão enxergue gargalos, que o planejamento trabalhe com dados confiáveis e que o chão de fábrica registre a produção de forma simples, reduzindo atrasos, perdas de informação e retrabalho administrativo.

2. Análise do Sistema Proposto

2.1 Escopo do Sistema

O LumiFlow é um sistema web, acessado pelo navegador, que controla o fluxo de produção de uma indústria desde o cadastro da estrutura produtiva até a conclusão das ordens de produção. A interface é renderizada no servidor e as regras de negócio ficam concentradas em uma camada de serviços, o que garante que todas as operações sigam as mesmas validações.

Estrutura produtiva: o sistema permite cadastrar os setores da fábrica (por exemplo, Corte, Dobra, Solda, Pintura e Montagem), as etapas internas de cada setor (como 1ª e 2ª Dobra), as máquinas instaladas em cada setor e os operadores, que podem ter um setor padrão ou atuar como volantes.

Produtos e roteiros: cada produto é identificado por um código e vinculado a um roteiro de fabricação, que define a sequência de setores e etapas por onde ele passa. Os roteiros são reutilizáveis por vários produtos e podem ser duplicados para criar novas versões; um roteiro já usado em ordens não pode ter sua estrutura alterada, o que preserva o histórico das ordens.

Ordens de produção: o PCP abre ordens informando produto, quantidade e observações. O sistema gera o número da ordem automaticamente (OP-0001, OP-0002...), grava o roteiro vigente e cria a primeira etapa no primeiro setor do roteiro. A gestão libera as etapas para produção, pode pausá-las e pode dividi-las em variantes (por exemplo, por cor), desde que ainda não tenham produção lançada.

Lançamentos no chão de fábrica: em cada etapa liberada, o operador registra a quantidade produzida, a máquina e o operador responsável. O sistema atualiza as quantidades produzida e pendente, conclui a etapa quando o pendente chega a zero e encaminha automaticamente a quantidade para a etapa seguinte do roteiro. Quando todas as etapas são concluídas, a ordem passa para o status Concluída.

Qualidade: peças com defeito são registradas como refugo, com motivo, setor de origem e destino. Quando o destino é retrabalho, o sistema cria uma linha de retrabalho no setor de origem; quando é descarte, as peças saem da produção. O retrabalho realizado é registrado e vinculado ao refugo correspondente.

Estoque de vidro: o sistema mantém o cadastro dos tipos de chapa de vidro com estoque mínimo, registra entradas e consumos (saídas) e calcula o saldo a partir do histórico de movimentações, alertando quando o saldo fica abaixo do mínimo e impedindo saídas maiores que o estoque.

Acompanhamento e gestão: o dashboard apresenta indicadores (etapas ativas, em andamento, concluídas no dia e peças aguardando retrabalho) e a lista de ordens com progresso e etapa atual. Também há visão das ordens por setor, detalhe completo de cada ordem com linha do tempo e histórico de lançamentos com filtros.

Segurança e acesso: o acesso exige login e senha. Os usuários têm perfis hierárquicos (ADMIN, SUPERVISOR, PCP, GESTAO e OPERADOR) e cada tela e operação é autorizada no servidor conforme o perfil. A interface está disponível em português e inglês e oferece tema claro, escuro e identidade visual.

2.2 Não Escopo do Sistema

O LumiFlow concentra-se no controle do fluxo produtivo. Ficam fora do escopo:

- Não serão desenvolvidos módulos de vendas, faturamento, emissão de notas fiscais ou controle financeiro;
- Não será feito o cálculo de custos de produção nem a formação de preço dos produtos;

- Não haverá controle de compras, fornecedores ou estoque de matérias-primas além das chapas de vidro;
- Não haverá planejamento automático de capacidade (MRP/APS) nem sequenciamento automático das ordens;
- Não será feito o apontamento de tempos de máquina e de paradas; a tela de relatório de tempo é apenas um protótipo visual;
- Não serão desenvolvidos aplicativo móvel nativo, integração com ERPs externos ou envio de notificações por e-mail;
- Não serão emitidos relatórios em PDF ou planilha; as consultas são feitas nas próprias telas do sistema.

2.3 Requisitos do Sistema

Requisitos funcionais (RF) e não funcionais (RNF) do sistema:

Nr.	Requisito
RF01	O sistema deve autenticar os usuários por login e senha e bloquear temporariamente o acesso após falhas consecutivas.
RF02	O sistema deve permitir o cadastro (inclusão, alteração e exclusão) de usuários com perfil de acesso, somente pelo ADMIN.
RF03	O sistema deve permitir o cadastro de setores e de suas etapas internas, em ordem sequencial.
RF04	O sistema deve permitir o cadastro de máquinas vinculadas a um setor.
RF05	O sistema deve permitir o cadastro de operadores, com função e setor padrão opcional.
RF06	O sistema deve permitir o cadastro de roteiros de fabricação, com passos ordenados por setor e etapa, além de duplicá-los e vinculá-los a produtos.
RF07	O sistema deve impedir a alteração da estrutura e a exclusão de roteiros já utilizados em ordens de produção.
RF08	O sistema deve permitir o cadastro de produtos identificados por código e vinculados a um roteiro.
RF09	O sistema deve permitir a abertura de ordens de produção, gerando o número automaticamente e gravando o roteiro vigente do produto.
RF10	O sistema deve permitir liberar, pausar e dividir em variantes as etapas de uma ordem.
RF11	O sistema deve registrar o lançamento de produção informando quantidade, máquina e operador, sem permitir quantidade maior que a pendente.
RF12	O sistema deve encaminhar automaticamente a quantidade produzida para a etapa seguinte do roteiro e concluir a ordem quando todas as etapas forem concluídas.
RF13	O sistema deve registrar refugos com motivo, setor de origem e destino (retrabalho ou descarte) e registrar o retrabalho realizado.
RF14	O sistema deve exibir um dashboard com indicadores de produção, a lista de ordens com progresso e a visão de ordens por setor.
RF15	O sistema deve exibir o detalhe de cada ordem, com etapas, roteiro e histórico de eventos, e o histórico geral de lançamentos com filtros.
RF16	O sistema deve controlar o estoque de chapas de vidro, registrando entradas e saídas, calculando o saldo e alertando o estoque mínimo.
RNF01	O sistema deve controlar o acesso por perfil com hierarquia (ADMIN > SUPERVISOR > PCP/GESTAO > OPERADOR), validado no servidor.
RNF02	As senhas devem ser armazenadas com hash BCrypt e as operações que alteram dados devem ser protegidas contra CSRF.
RNF03	O sistema deve impedir que edições simultâneas sobrescrevam dados (controle de concorrência otimista e pessimista).
RNF04	A interface deve estar disponível em português e inglês e ser responsiva para desktop, tablet e celular.
RNF05	O banco de dados deve ser PostgreSQL, com versionamento de estrutura por migrações Flyway.
RNF06	O sistema deve ser empacotado em contêiner Docker e implantado em nuvem, com verificação de saúde da aplicação.
RNF07	Toda alteração de código deve passar por integração contínua, com compilação e testes automatizados.

2.4 Glossário

Termo	Descrição
Ordem de produção (OP)	Pedido interno para fabricar uma quantidade de um produto, identificado por um número sequencial (ex.: OP-0001).
Roteiro de fabricação	Sequência de passos (setor e etapa) pela qual um produto passa até ficar pronto.
Passo do roteiro	Cada item do roteiro, com número de sequência, setor e, opcionalmente, etapa.
Setor	Área da fábrica responsável por um tipo de operação, como Corte, Dobra, Solda, Pintura e Montagem.
Etapa do setor	Subdivisão sequencial de um setor, como 1ª Dobra e 2ª Dobra.
Linha da ordem (ordem-setor)	Andamento da ordem em um setor: quantidades recebida, produzida e pendente, e status.
Lançamento	Registro da quantidade produzida em uma etapa, com máquina e operador.
Refugo	Peça com defeito retirada da produção, com motivo, setor de origem e destino.
Retrabalho	Correção de peças refugadas, feita no setor onde o defeito se originou.
Descarte	Destino do refugo em que a peça não pode ser recuperada e sai da produção.
Variante	Divisão de uma etapa em partes com características diferentes, como cores.
PCP	Planejamento e Controle da Produção: perfil responsável por cadastros, roteiros e abertura de ordens.
Liberar etapa	Autorizar que uma etapa disponível comece a receber lançamentos de produção.
Status da etapa	Situação da linha: Disponível, Liberada, Em andamento ou Concluída.
Chapa de vidro	Insumo controlado em estoque, cadastrado por tipo (incolor, jateado ou reflexivo).
Estoque mínimo	Quantidade de chapas abaixo da qual o sistema emite alerta de reposição.
Operador volante	Operador sem setor padrão, que pode atuar em qualquer setor.

2.5 Mapeamento de Processos

O fluxo abaixo representa o processo produtivo atendido pelo LumiFlow, em notação inspirada no BPMN, com raias para cada participante: o PCP prepara os cadastros e abre a ordem; a gestão decide se divide a etapa em variantes e a libera; o operador lança a produção e registra refugos e retrabalhos; e o sistema encaminha as quantidades para a etapa seguinte do roteiro até concluir a ordem.

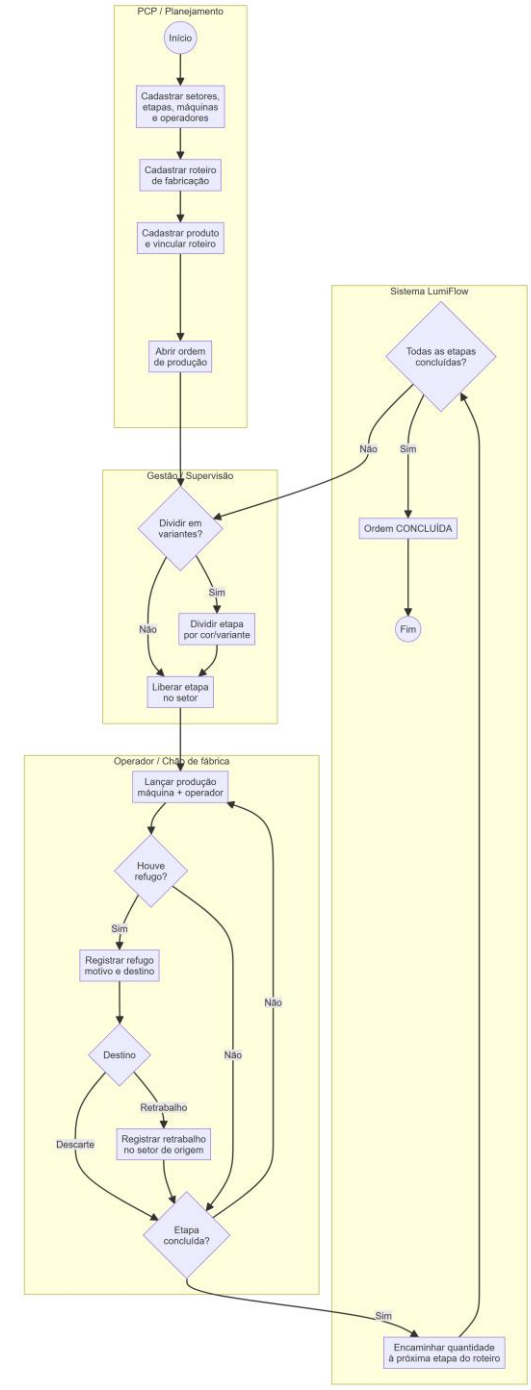


Figura 1 – Processo produtivo atendido pelo LumiFlow

2.6 Diagrama de Caso de Uso

O diagrama apresenta os atores do LumiFlow e os casos de uso disponíveis a cada perfil. Como os perfis são hierárquicos, um perfil superior herda os casos de uso dos perfis abaixo dele (linhas tracejadas): o SUPERVISOR herda PCP e GESTÃO, que por sua vez herdam OPERADOR, e o ADMIN herda todos.

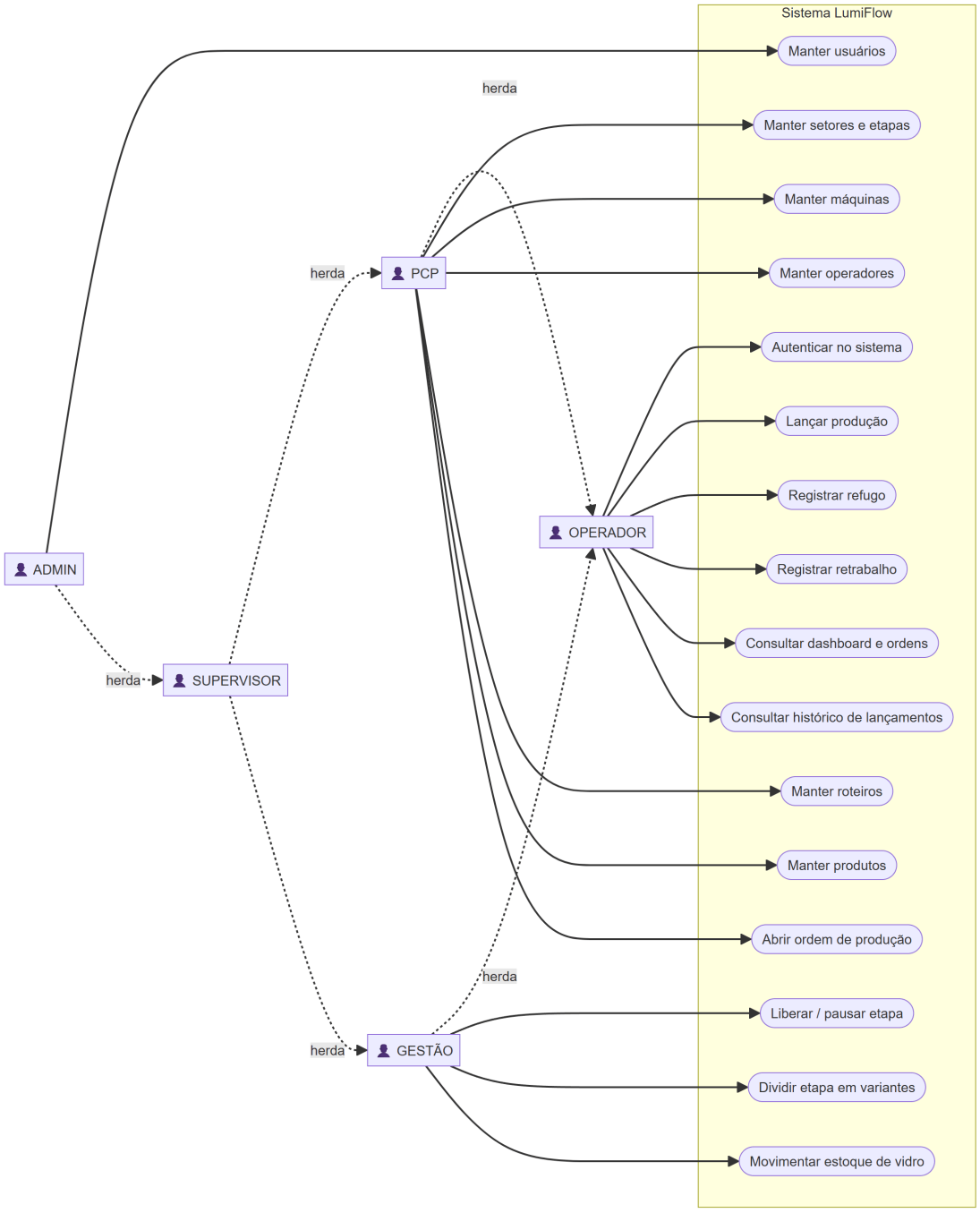


Figura 2 – Diagrama de casos de uso do LumiFlow

2.7 Caso de Uso Expandido

Caso de uso 01: Abrir ordem de produção

Campo	Descrição
Nome do caso de uso	Abrir ordem de produção
Grau de importância	Alta
Ator principal	PCP
Atores secundários	SUPERVISOR, ADMIN
Resumo	O PCP abre uma ordem de produção para um produto, informando a quantidade. O sistema gera o número, grava o roteiro vigente do produto e cria a primeira etapa da ordem.
Pré-condições	Usuário autenticado com perfil PCP ou superior; produto cadastrado e vinculado a um roteiro com ao menos um passo.
Pós-condições	Ordem criada com status Disponível e primeira etapa disponível no primeiro setor do roteiro.

Ações do ator	Ações do sistema
1. Acessa a opção Nova Ordem.	2. Exibe o formulário de nova ordem.
3. Pesquisa e seleciona o produto.	4. Exibe o roteiro do produto (etapas).
5. Informa quantidade, data e observação e confirma.	6. Valida os dados e o roteiro do produto.
	7. Gera o número sequencial (OP-0001...), grava a ordem e a primeira etapa.
	8. Exibe a mensagem "Ordem de produção criada com sucesso!".

Campo	Descrição
Fluxos alternativos	A1. Produto sem roteiro: o sistema exibe "Este produto não possui roteiro de produção cadastrado." e não cria a ordem. A2. Quantidade inválida: o sistema informa que a quantidade deve ser maior que zero.
Restrições	Somente perfis PCP, SUPERVISOR e ADMIN podem abrir ordens.

Caso de uso 02: Lançar produção

Campo	Descrição
Nome do caso de uso	Lançar produção
Grau de importância	Alta
Ator principal	OPERADOR
Atores secundários	Supervisor, gestão
Resumo	O operador registra a quantidade produzida em uma etapa liberada, informando a máquina e o operador. O sistema atualiza as quantidades e encaminha as peças para a etapa seguinte do roteiro.
Pré-condições	Usuário autenticado; etapa da ordem com status Liberada ou Em andamento e quantidade pendente maior que zero.
Pós-condições	Lançamento registrado, quantidades atualizadas e quantidade encaminhada à próxima etapa; ordem concluída quando todas as etapas terminam.

Ações do ator	Ações do sistema
1. Acessa as ordens do seu setor e escolhe a etapa.	2. Exibe a tela de lançamento com a quantidade máxima pendente.
3. Informa quantidade, máquina, operador e observação.	4. Valida a quantidade e se máquina e operador pertencem ao setor.
5. Confirma o lançamento no resumo.	6. Registra o lançamento e atualiza produzido e pendente.
	7. Muda a etapa para Em andamento ou Concluída e encaminha a quantidade à próxima etapa.
	8. Exibe "Lançamento registrado com sucesso!".

Campo	Descrição
Fluxos alternativos	A1. Quantidade maior que a pendente: o sistema exibe "A quantidade lançada não pode ser maior que a pendente." A2. Peças com defeito: o operador usa a aba Refugo, informa motivo, setor de origem e destino. A3. Linha de retrabalho: o lançamento deve ser feito na aba Retrabalho.
Restrições	Etapas Disponíveis (não liberadas) ou Concluídas não recebem lançamentos.

2.8 Diagrama de Classe

O diagrama de classes mostra as entidades persistidas do LumiFlow (pacote entity), seus atributos principais e as associações com multiplicidade. A classe abstrata Auditoria fornece as datas de criação e de atualização às demais entidades.

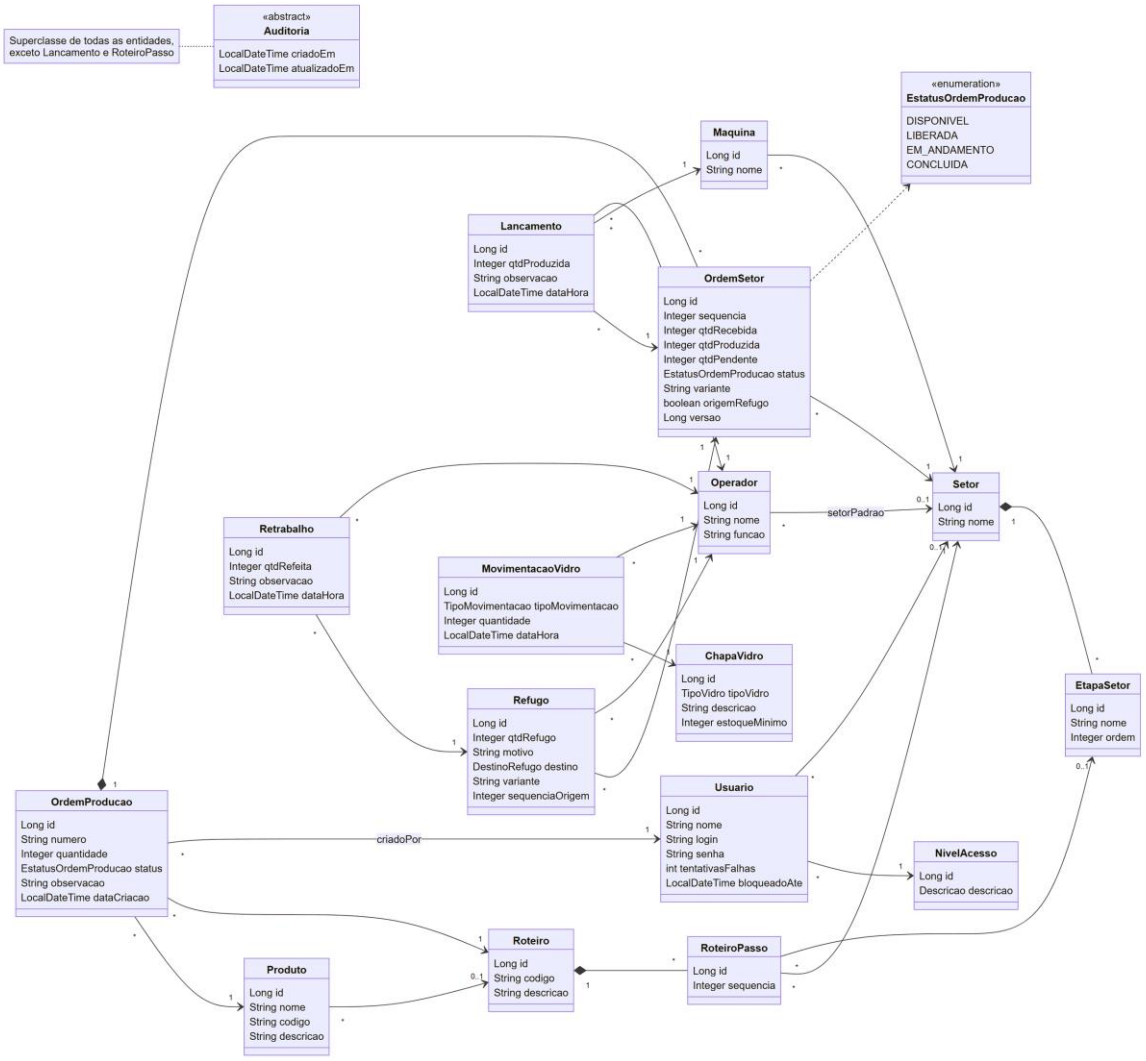


Figura 3 – Diagrama de classes das entidades do LumiFlow

2.9 Testes do Software/Aplicativo/Solução

1. Plano de Teste

Objetivo: garantir que as regras de negócio, as telas e o controle de acesso do LumiFlow funcionem conforme os requisitos, evitando regressões a cada alteração do código.

Abordagem: testes automatizados de unidade e de integração da camada web, escritos com JUnit 5, Mockito, AssertJ, MockMvc e Spring Security Test. A suíte cobre controllers, serviços, mapeadores, DTOs, validações, tratamento de erros, internacionalização e a matriz completa de autorização por perfil.

Ambiente: a suíte é executada a cada envio de código por integração contínua (GitHub Actions), com um banco PostgreSQL 18 descartável e o perfil de produção, o que também valida as migrações Flyway contra um banco limpo. A implantação no Railway só ocorre depois que a integração contínua é aprovada.

Resultado atual: 1.140 testes automatizados executados, sem falhas.

2. Casos de Teste

ID	Cenário	Entrada	Resultado esperado	Situação
CT01	Lançar quantidade maior que a pendente	Etapa com 37 peças pendentes; lançamento de 40	Lançamento rejeitado com a mensagem de quantidade excedente	Aprovado
CT02	Lançar exatamente o pendente	Etapa com 10 pendentes; lançamento de 10	Etapa concluída, data de fim registrada e quantidade enviada à próxima etapa	Aprovado
CT03	Abrir ordem de produto sem roteiro	Produto sem roteiro vinculado	Ordem não criada; mensagem "produto sem roteiro"	Aprovado
CT04	Concluir ordem	Última etapa aberta concluída	Ordem passa para o status Concluída	Aprovado
CT05	Saída de vidro sem estoque	Saldo de 6 chapas; saída de 10	Movimentação rejeitada por estoque insuficiente	Aprovado
CT06	Excluir o último administrador	Único usuário ADMIN	Exclusão bloqueada	Aprovado
CT07	Bloqueio de login	Falhas consecutivas de senha	Acesso bloqueado temporariamente, de forma escalonada	Aprovado
CT08	Acesso por perfil	OPERADOR acessa a lista geral de ordens	Resposta 403 (acesso negado)	Aprovado
CT09	Redirecionamento seguro	Erro com cabeçalho Referer de outro site	Usuário redirecionado para página interna (sem open redirect)	Aprovado
CT10	Cadastro duplicado	Produto com código já existente	Cadastro rejeitado e nada é gravado	Aprovado

3. Projeto de Interface Gráfica (Modelo não Funcional)

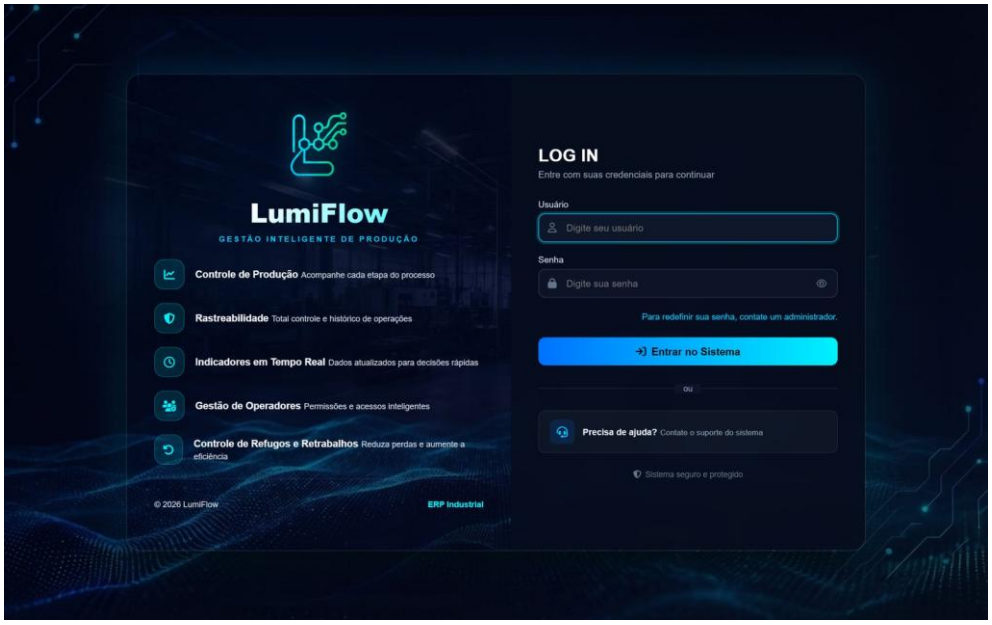
3.1 Modelo Não Funcional
As telas a seguir foram capturadas do sistema em funcionamento, com dados de demonstração. Todas compartilham o menu lateral, que exibe apenas as opções permitidas ao perfil do usuário, e se adaptam a telas de celular e tablet.

Interface 01: Tela de login

Descrição: Responsável pela autenticação dos usuários no sistema.

Nome: Login.html (/login)

Caso de uso: Autenticar no sistema

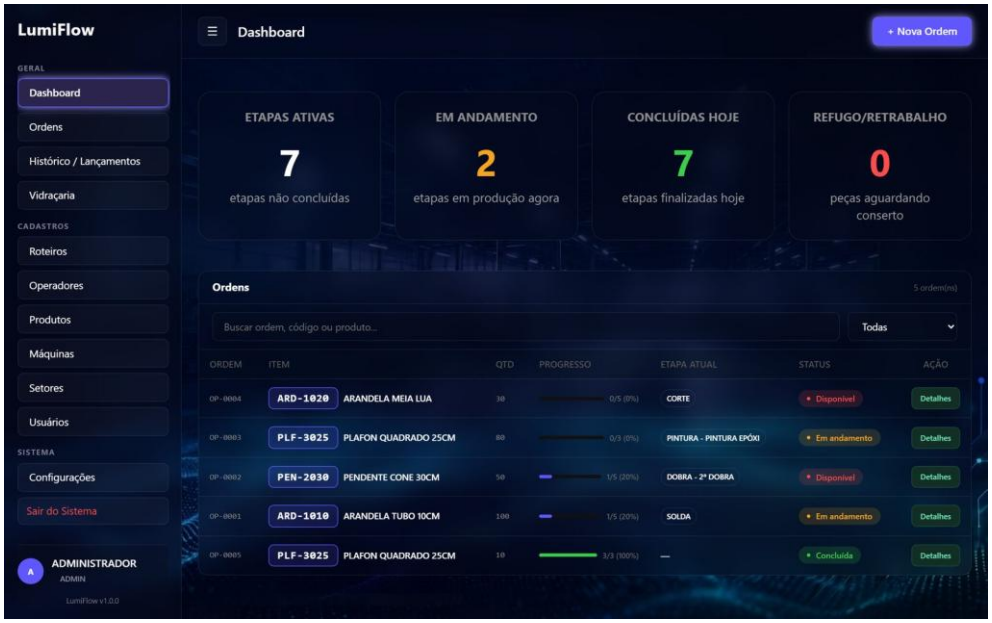


Interface 02: Dashboard

Descrição: Apresenta os indicadores de produção e a lista de ordens com progresso, etapa atual e status.

Nome: Dashboard.html (/dashboard)

Caso de uso: Consultar dashboard e ordens

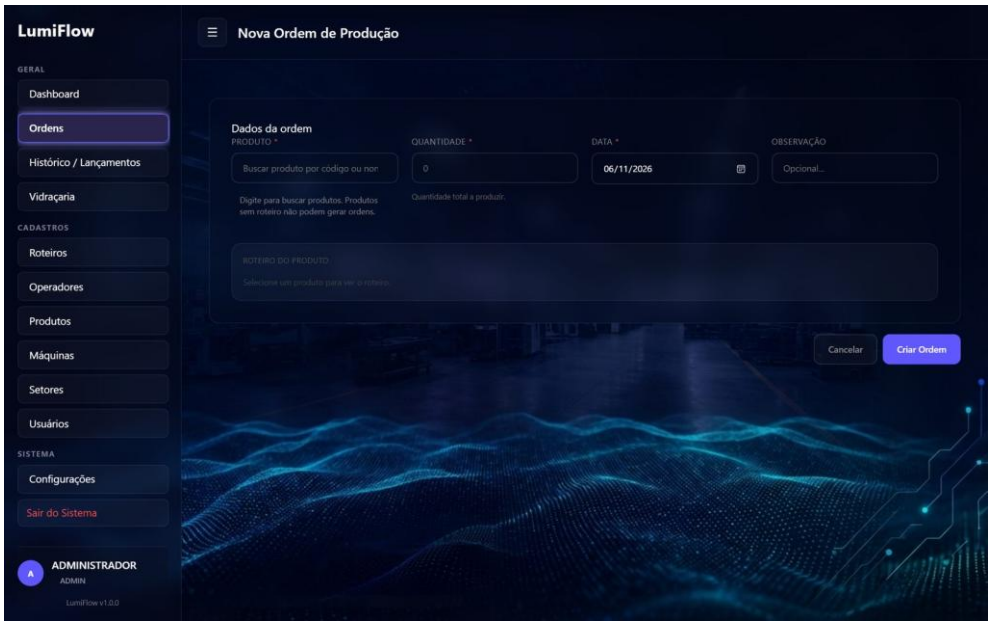


Interface 03: Nova ordem de produção

Descrição: Responsável pela abertura de ordens, com busca do produto e exibição do seu roteiro.

Nome: NovaOrdem.html (/dashboard/ordens/novaOrdem)

Caso de uso: Abrir ordem de produção

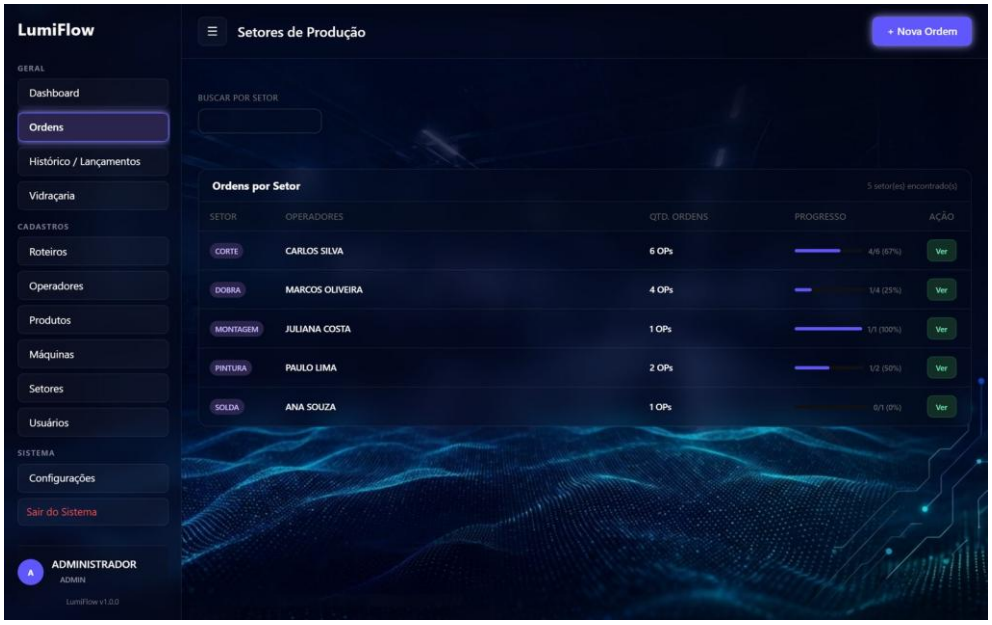


Interface 04: Ordens por setor

Descrição: Resume as ordens de cada setor e dá acesso à lista de ordens do setor.

Nome: ListaOrdemSetores.html (/dashboard/ordens-setor)

Caso de uso: Consultar dashboard e ordens

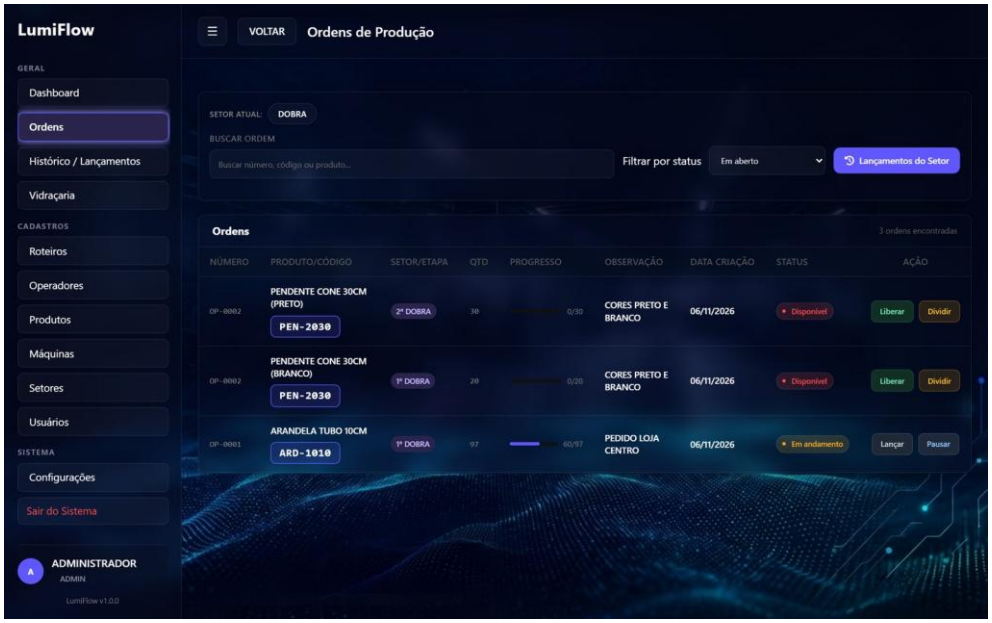


Interface 05: Ordens do setor

Descrição: Lista as etapas das ordens no setor, com ações de liberar, dividir, pausar e lançar.

Nome: ListaOrdem.html (/dashboard/ordens-setor/{setor})

Caso de uso: Liberar / pausar etapa; Dividir etapa em variantes

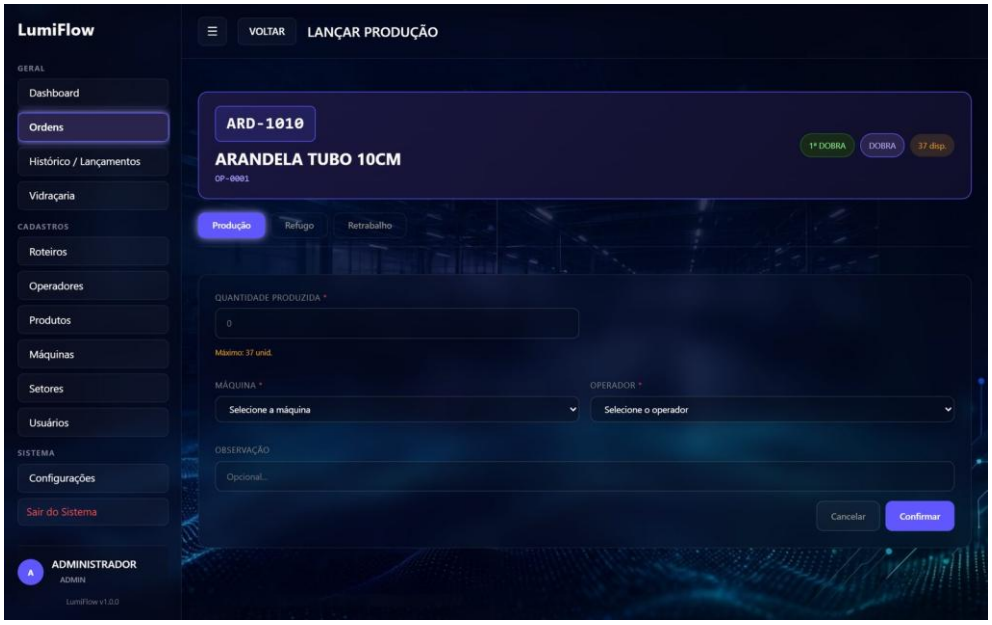


Interface 06: Lançar produção

Descrição: Registra produção, refugo e retrabalho de uma etapa, em abas.

Nome: Lancamento.html (/dashboard/ordens/{ordem}/setor/{etapa}/lancar)

Caso de uso: Lançar produção; Registrar refugo; Registrar retrabalho

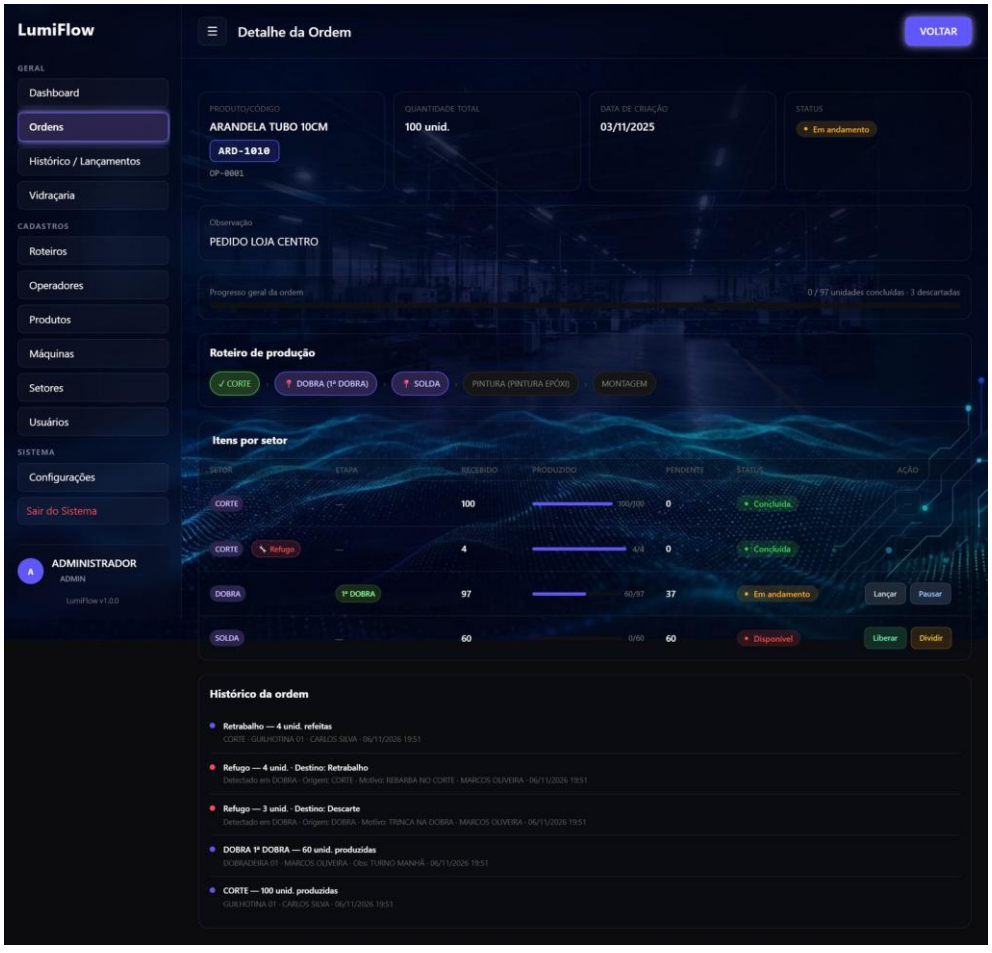


Interface 07: Detalhe da ordem

Descrição: Mostra os dados da ordem, o roteiro, as etapas por setor e a linha do tempo de eventos.

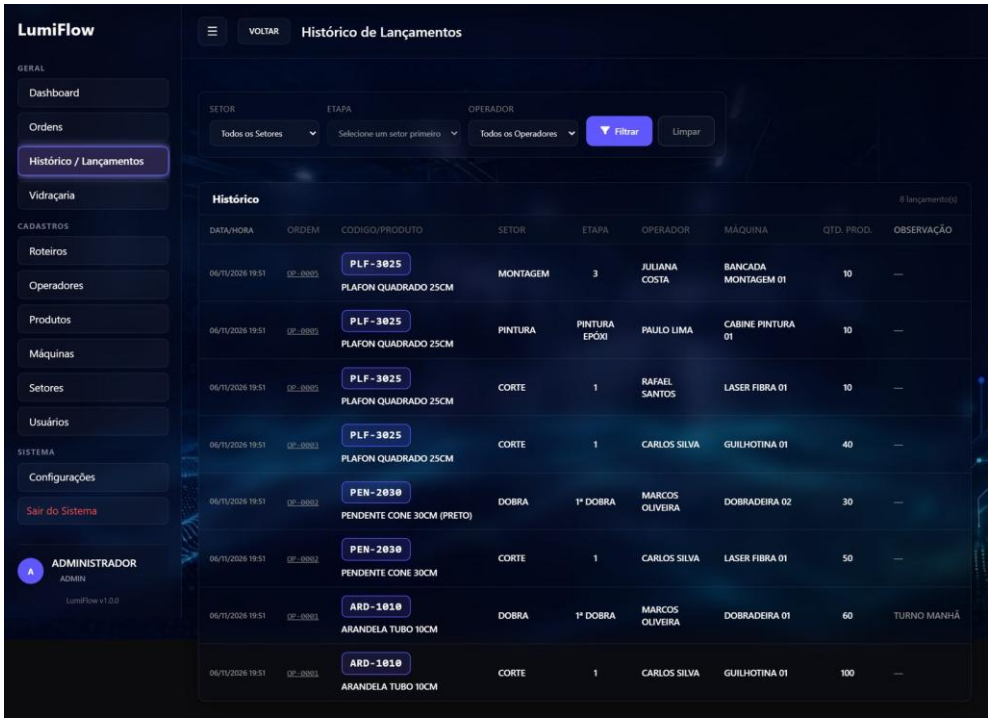
Nome: DetalhesOrdem.html (/dashboard/ordens/{id}/detalhes)

Caso de uso: Consultar dashboard e ordens



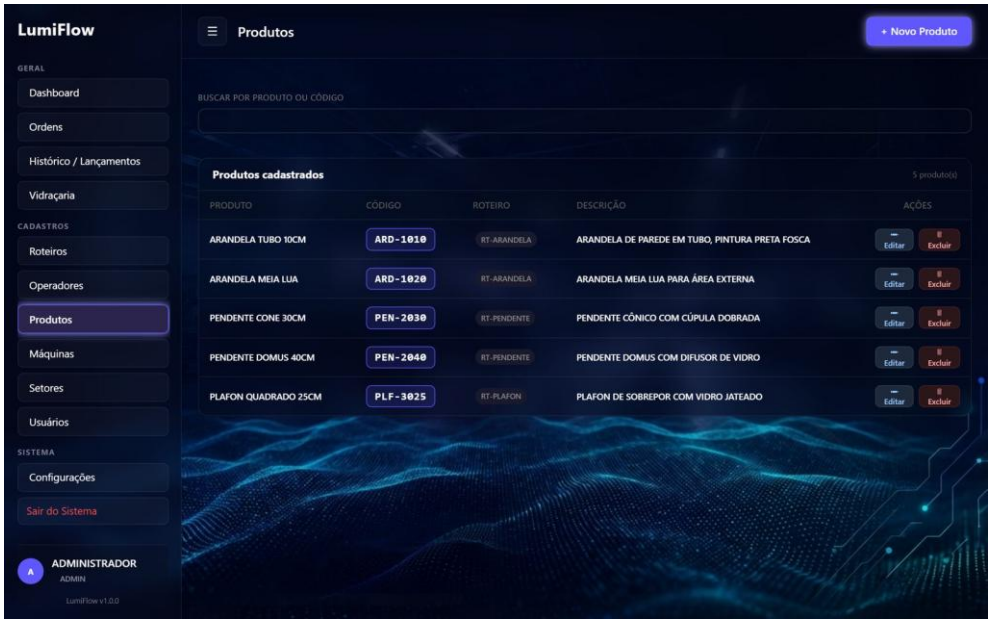
Interface 08: Histórico de lançamentos

Descrição: Lista todos os lançamentos de produção, refugo e retrabalho, com filtros.
Nome: HistoricoLancamento.html (/dashboard/historico)
Caso de uso: Consultar histórico de lançamentos



Interface 09: Cadastro de produtos

Descrição: Permite incluir, alterar, excluir e pesquisar produtos e vinculá-los a roteiros.
Nome: Produtos.html (/dashboard/produtos)
Caso de uso: Manter produtos

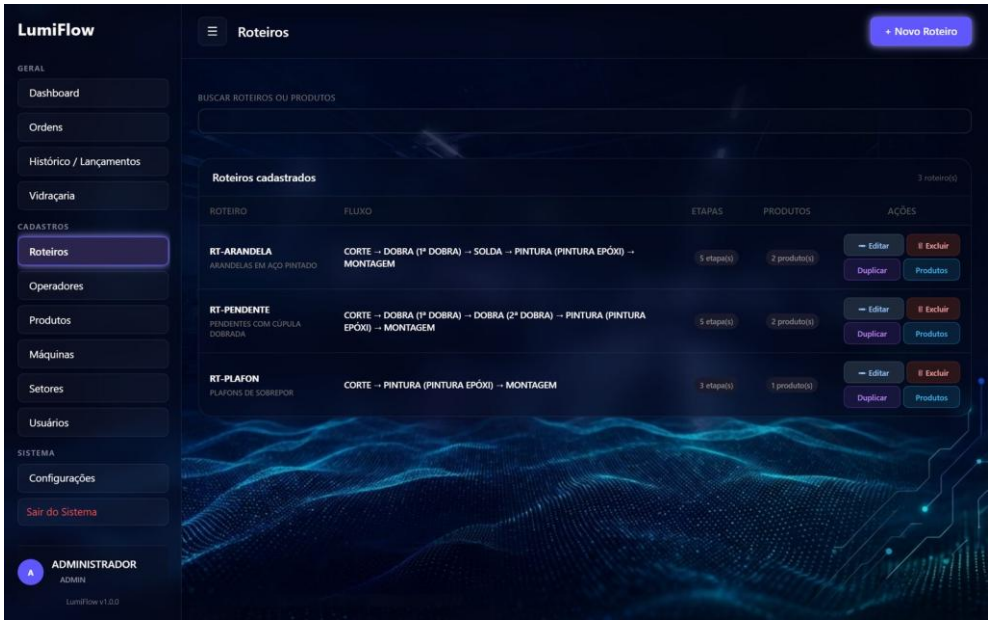


Interface 10: Cadastro de roteiros

Descrição: Permite criar, editar, duplicar e excluir roteiros e vincular produtos.

Nome: Roteiros.html (/dashboard/roteiros)

Caso de uso: Manter roteiros

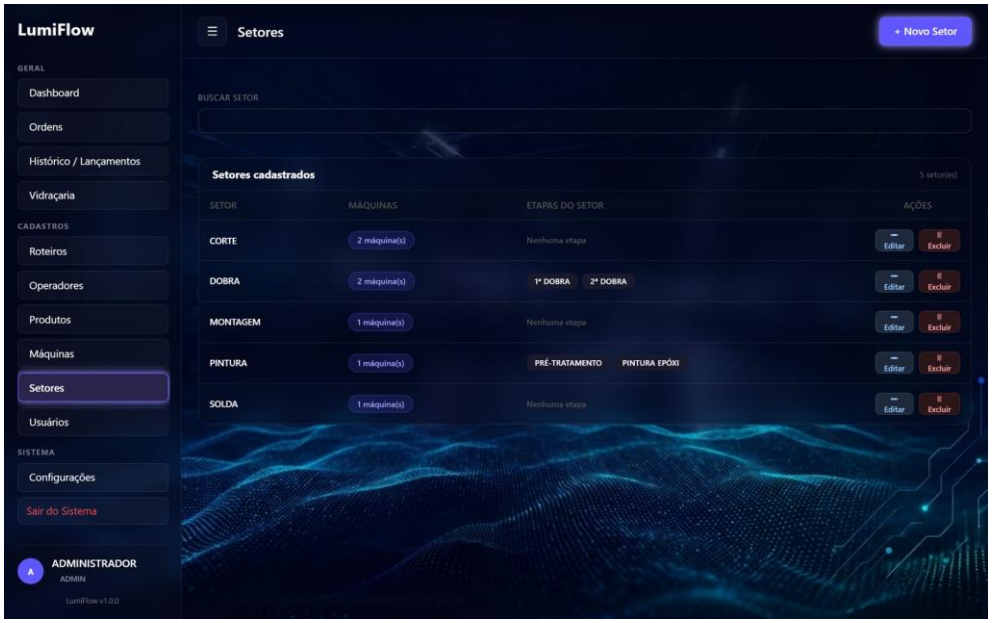


Interface 11: Cadastro de setores

Descrição: Permite manter os setores e suas etapas.

Nome: Setores.html (/dashboard/setores)

Caso de uso: Manter setores e etapas

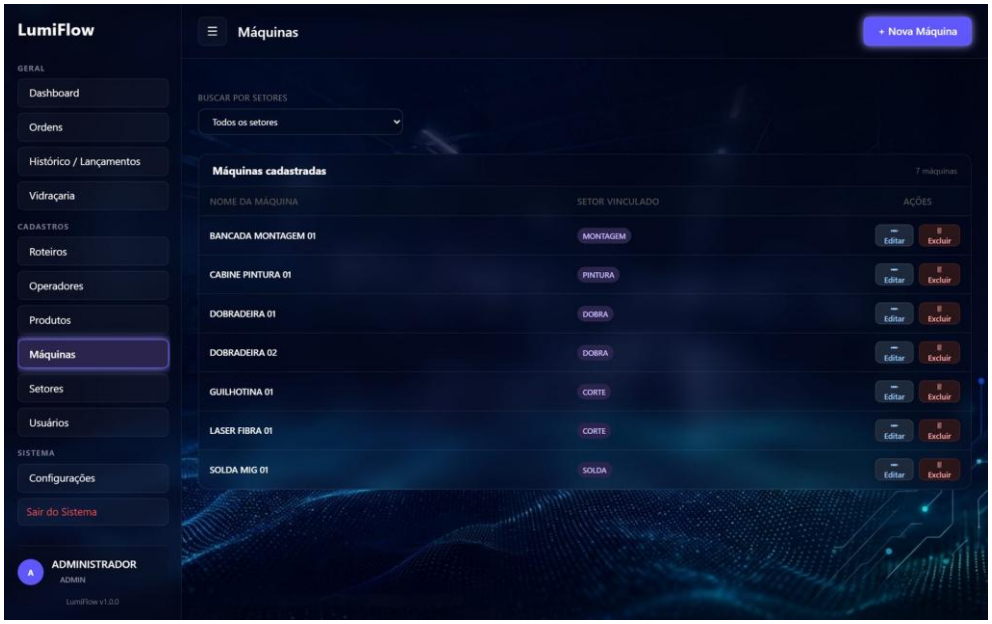


Interface 12: Cadastro de máquinas

Descrição: Permite manter as máquinas de cada setor.

Nome: CadastroMaquinas.html (/dashboard/maquinas)

Caso de uso: Manter máquinas

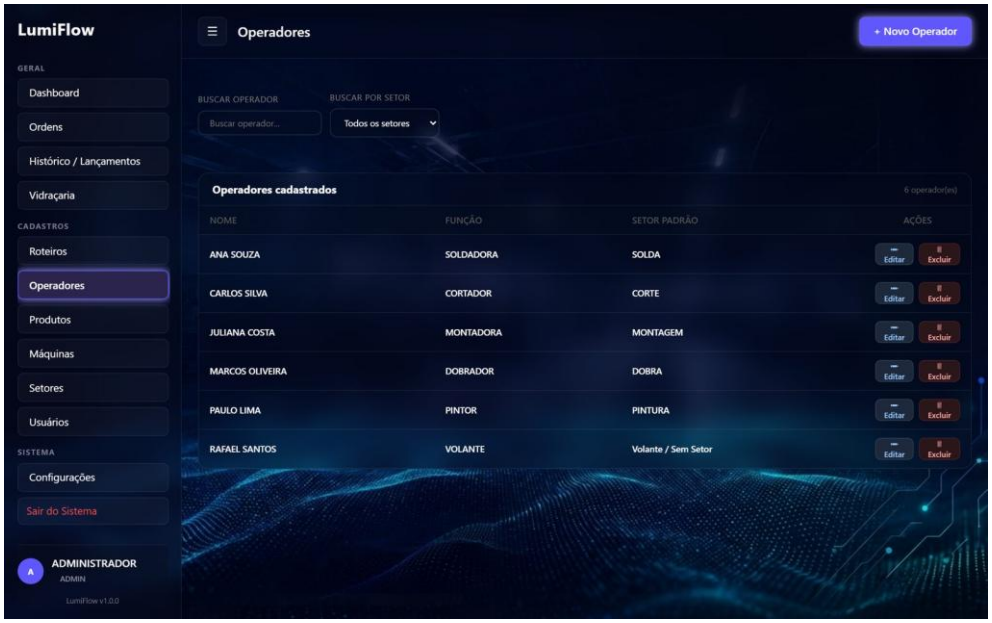


Interface 13: Cadastro de operadores

Descrição: Permite manter os operadores, sua função e setor padrão.

Nome: Operadores.html (/dashboard/operadores)

Caso de uso: Manter operadores

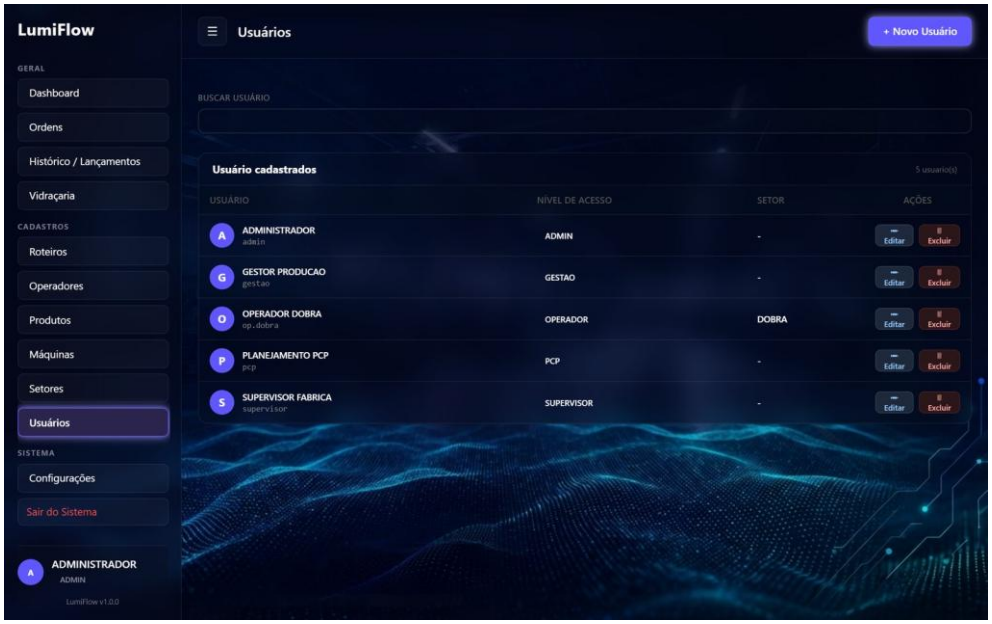


Interface 14: Cadastro de usuários

Descrição: Permite ao ADMIN manter os usuários e seus perfis de acesso.

Nome: ListaUsuario.html (/dashboard/usuario)

Caso de uso: Manter usuários

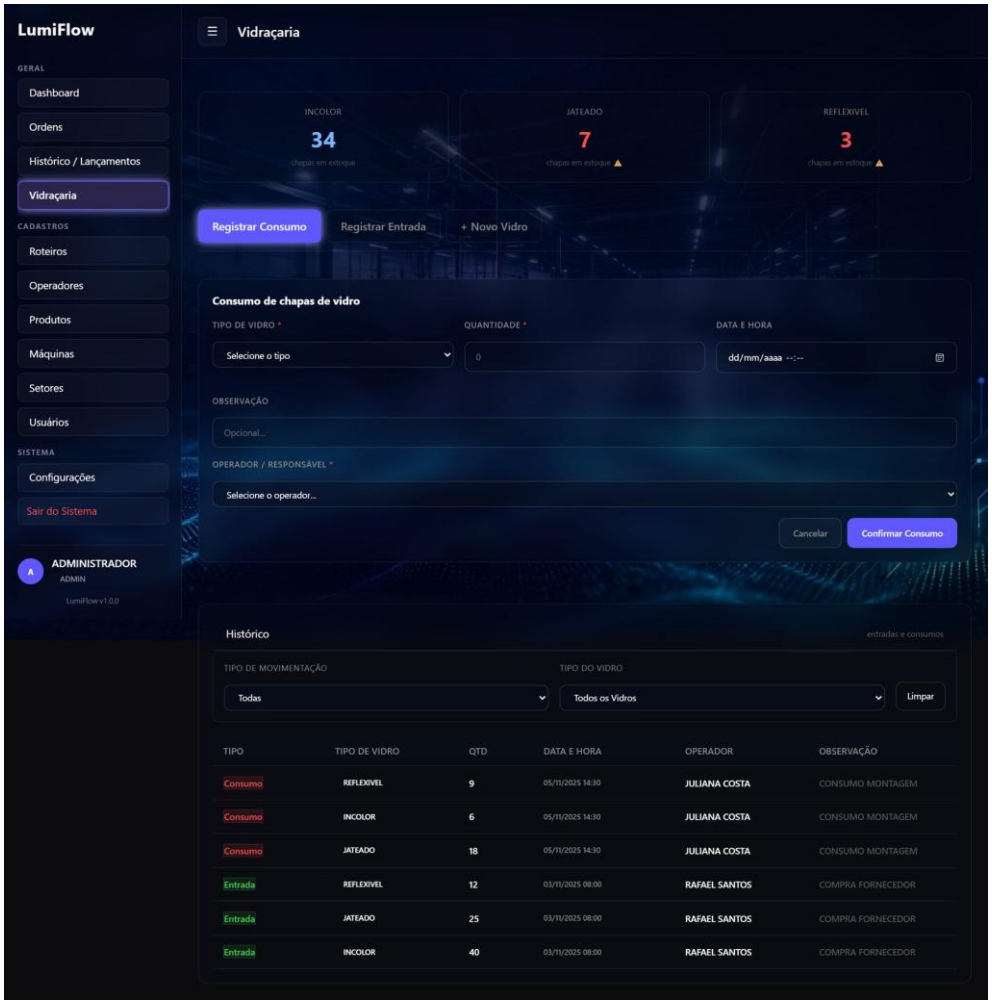


Interface 15: Vidraçaria

Descrição: Controla o estoque de chapas de vidro, com saldos, entradas, consumos e histórico.

Nome: Vidracaria.html (/dashboard/vidracaria)

Caso de uso: Movimentar estoque de vidro

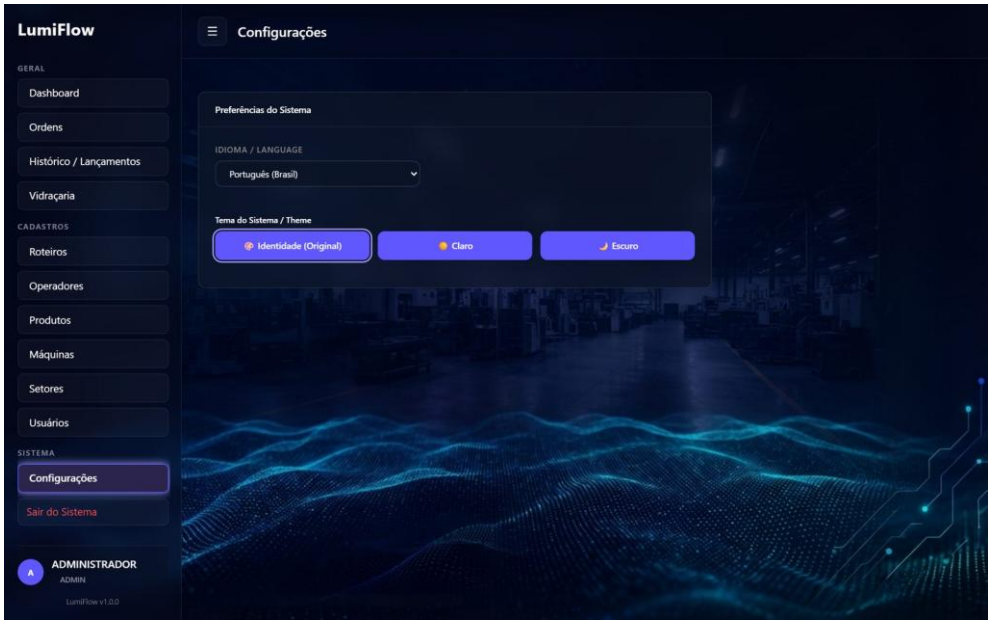


Interface 16: Configurações

Descrição: Permite escolher o idioma (português ou inglês) e o tema visual.

Nome: Configuracoes.html (/dashboard/configuracoes)

Caso de uso: —



3.2 Internacionalização do software

O LumiFlow é bilíngue (português do Brasil e inglês). As mensagens do servidor ficam nos arquivos messages.properties e messages_en.properties, e os textos da interface são traduzidos pelo dicionário do arquivo i18n.js. O idioma é escolhido na tela de Configurações ou pelo parâmetro ?lang=en na URL. A tabela abaixo apresenta uma amostra dos termos utilizados:

Português	Inglês
Dashboard	Dashboard
Ordens	Orders
Nova Ordem	New Order
Criar Ordem	Create Order
Roteiros	Production Routes
Produtos	Products
Máquinas	Machines
Setores	Departments
Operadores	Operators
Usuários	Users
Vidraçaria	Glassworks
Configurações	Settings
Sair do Sistema	Sign Out
Entrar no Sistema	Sign In
Etapas ativas	Active steps
Em andamento	In progress
Concluídas hoje	Completed today
Refugo/Retrabalho	Scrap/Rework
Disponível	Available
Concluída	Completed
Detalhes	Details
Dividir Ordem	Split Order
Confirmar	Confirm
Cancelar	Cancel
Descarte	Discard
Entrada	Receipt
Ordem de produção criada com sucesso!	Production order created successfully!
Lançamento registrado com sucesso!	Production entry registered successfully!
A quantidade lançada não pode ser maior que a pendente.	The entered quantity cannot exceed the pending quantity.
Estoque insuficiente para realizar esta saída.	Insufficient stock to complete this output.
Usuário ou senha inválidos.	Invalid username or password.

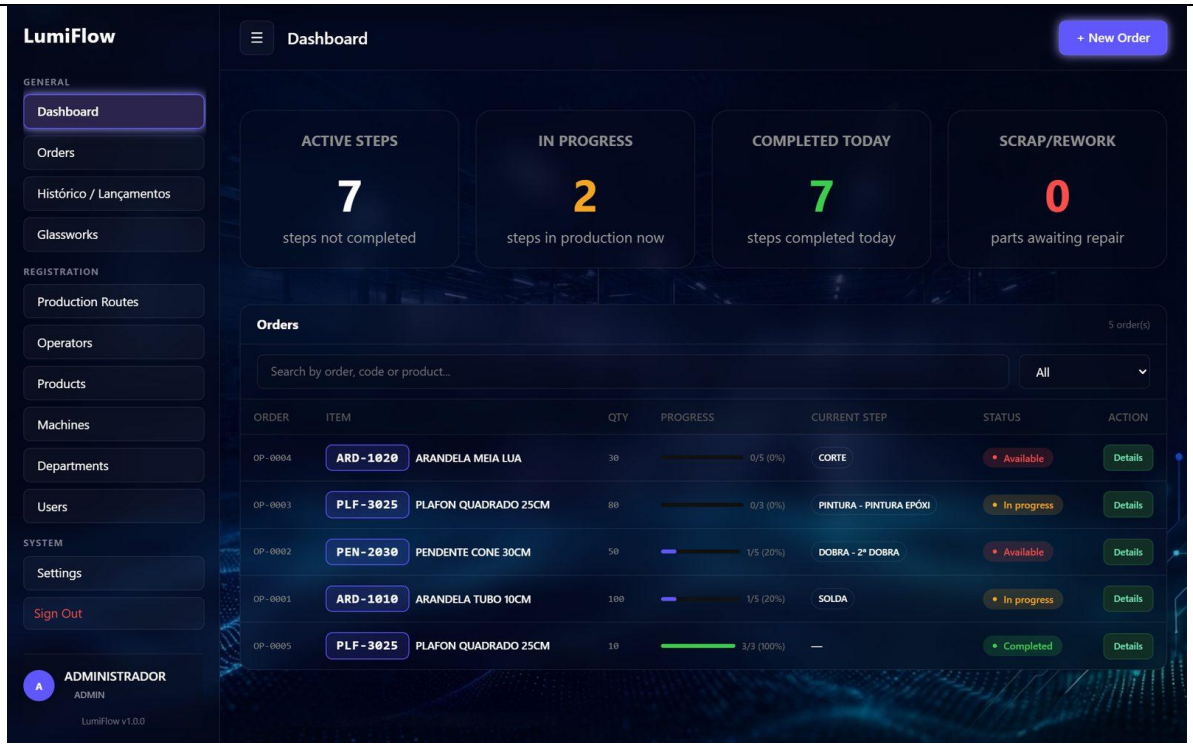


Figura 4 – Dashboard exibido em inglês (?lang=en)

4. Projeto de Banco de Dados

4.1 Diagrama de Entidade e Relacionamento (DER)
O modelo conceitual abaixo apresenta as 16 entidades do LumiFlow e seus relacionamentos, organizados em quatro grupos: segurança e acesso, estrutura de produção, ordens de produção e qualidade, e controle de vidros.

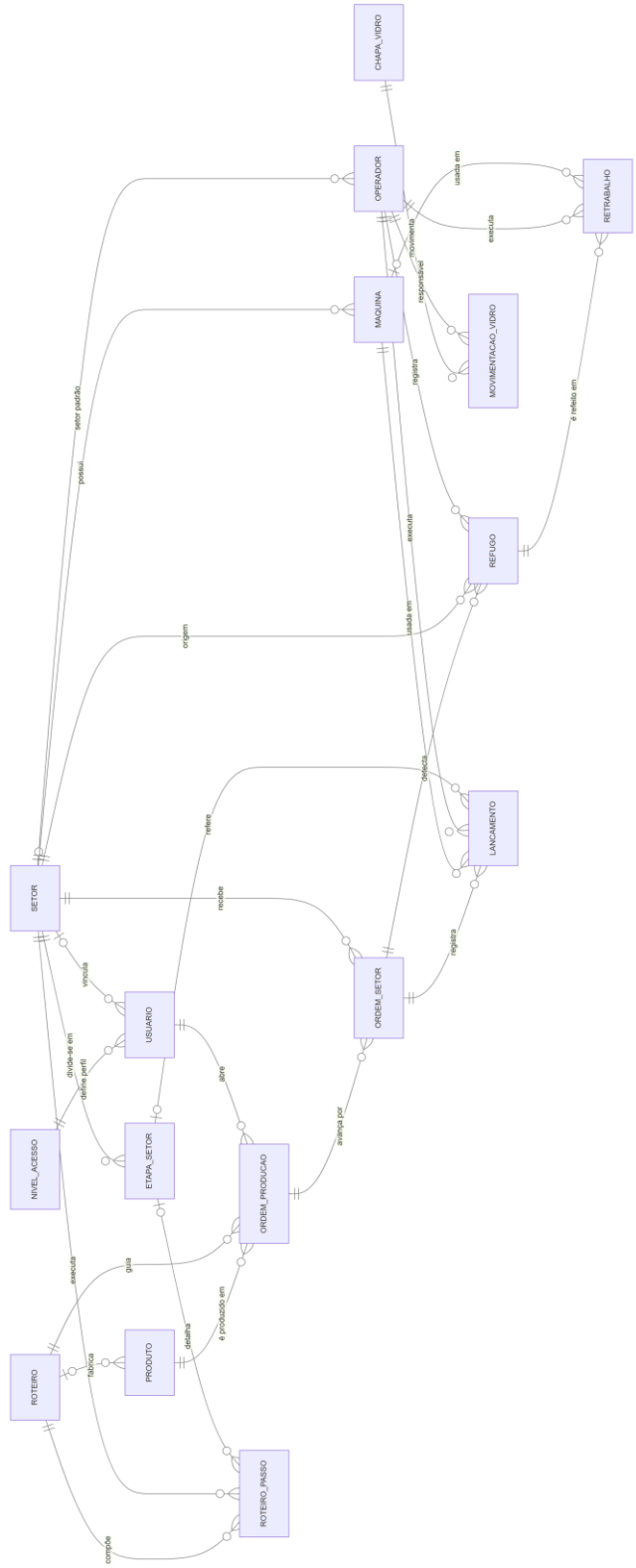


Figura 5 – Diagrama de entidade e relacionamento (notação pé de galinha)

4.2 Diagrama de Banco de Dados

Diagrama físico do banco PostgreSQL, com tipos, chaves primárias (PK), chaves estrangeiras (FK) e chaves únicas (UK). As colunas de auditoria criado_em e atualizado_em, presentes em todas as tabelas exceto roteiro_passo e lancamento, foram omitidas para facilitar a leitura.

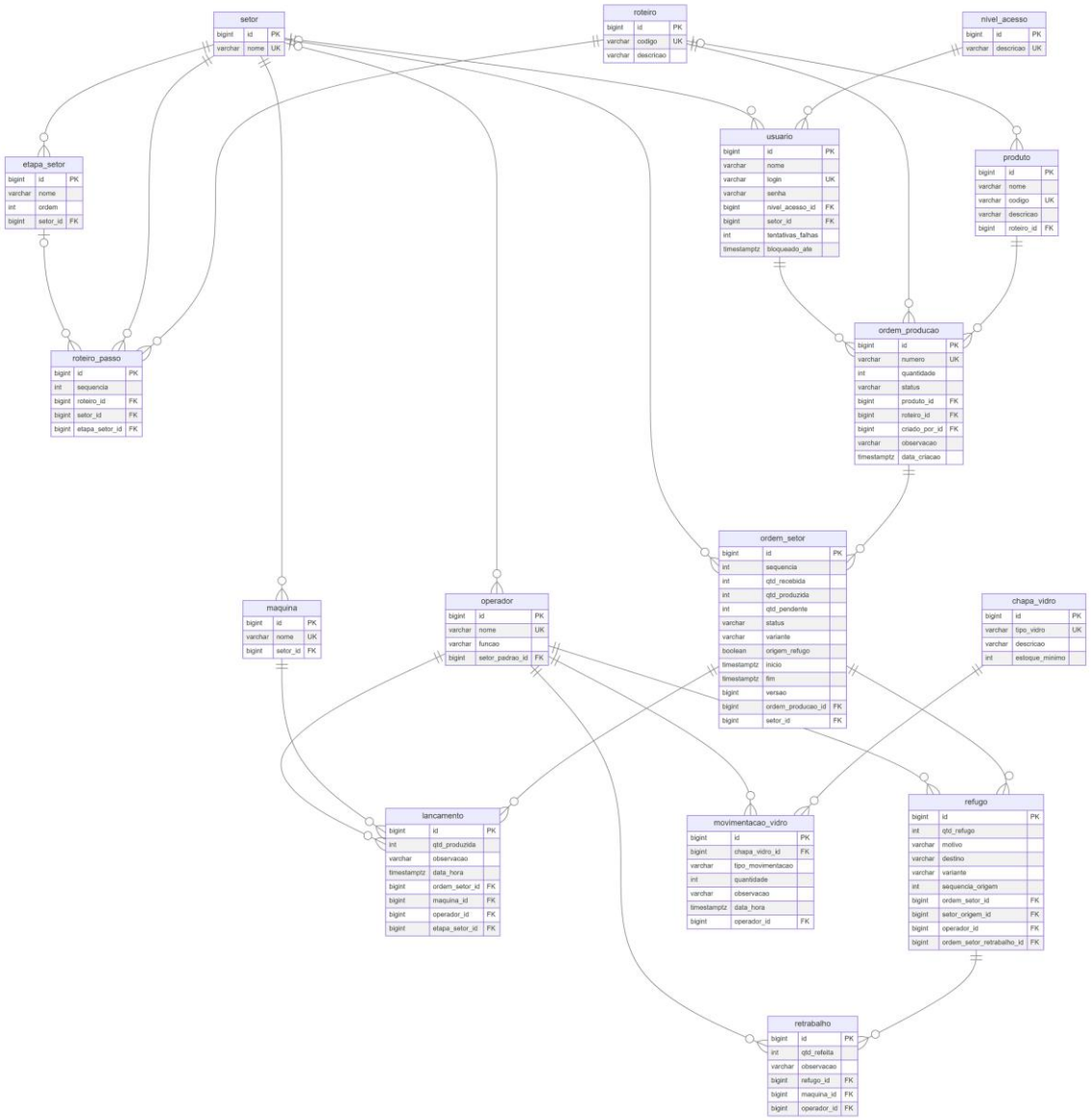


Figura 6 – Diagrama do banco de dados

4.3 Quadro Resumo dos Objetos de Banco de Dados

Objetos de banco de dados criados pelas migrações Flyway do LumiFlow:

Objeto de Banco de Dados	Quantidade
Tabelas	16
Views	0
Funções Escalares	0
Funções do Tipo Table	0
Procedures	0

Triggers	14
Funções de trigger	1
Índices de desempenho	34
Sequências (numeração das ordens)	1

O LumiFlow concentra as regras de negócio na camada de serviços da aplicação (Java/Spring), e as consultas são feitas pelos repositórios Spring Data JPA. Por isso, o banco utiliza apenas os objetos acima; views, funções e procedures não fazem parte do projeto real, conforme detalhado nas seções 4.7 a 4.10.

4.4 Mapeamento para o Modelo Lógico (Modelo Relacional)

Notação: atributos sublinhados (_id_) são chaves primárias e atributos com # são chaves estrangeiras.

nivel_acesso (id, descricao**).**

setor (id, nome**).**

usuario (id, nome, login, senha, #nivel_acesso_id, #setor_id, tentativas_falhas, bloqueado_ate**).**

operador (id, nome, funcao, #setor_padrao_id**).**

maquina (id, nome, #setor_id**).**

etapa_setor (id, nome, ordem, #setor_id**).**

roteiro (id, codigo, descricao**).**

roteiro_passo (id, sequencia, #roteiro_id, #setor_id, #etapa_setor_id**).**

produto (id, nome, codigo, descricao, #roteiro_id**).**

ordem_producao (id, numero, quantidade, status, #produto_id, #criado_por_id, observacao, data_criacao, #roteiro_id**).**

ordem_setor (id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente, status, variante, origem_refugo, inicio, fim, versao, #ordem_producao_id, #setor_id**).**

lancamento (id, qtd_produzida, observacao, data_hora, #ordem_setor_id, #maquina_id, #operador_id, #etapa_setor_id**).**

refugo (id, qtd_refugo, motivo, destino, variante, data_hora, sequencia_origem, #ordem_setor_id, #setor_origem_id, #operador_id, #ordem_setor_retrabalho_id**).**

retrabalho (id, qtd_refeita, observacao, data_hora, #refugo_id, #maquina_id, #operador_id**).**

chapa_vidro (id, tipo_vidro, descricao, estoque_minimo**).**

movimentacao_vidro (id, #chapa_vidro_id, tipo_movimentacao, quantidade, observacao, data_hora, #operador_id**).**

4.5 Script de Banco de Dados

O banco é criado e versionado pelas migrações Flyway, executadas automaticamente na inicialização da aplicação, em ordem de versão. Script completo das migrações V1 a V6:

V1__create_tables.sql

```
-- ============================================================================
-- V1 — ESTRUTURA DO BANCO: TABELAS, CHAVES, RELACIONAMENTOS E REGRAS (CHECK)
-- ============================================================================
-- Convenções de nomes:
--   fk_<tabela>_<referenciada> | uq_<tabela>_<coluna> | chk_<tabela>_<regra>
--
-- Regras:
--   * Constraints (PK, FK, UNIQUE, CHECK) ficam aqui, junto da tabela.
--   * Triggers e funções ficam na V2; índices de desempenho ficam na V3.
--   * Valores de CHECK espelham os enums Java. Ao criar um valor novo no enum,
--     atualize o CHECK correspondente em uma nova migration.
-- ============================================================================


-- ============================================================================
-- 1. SEGURANÇA E CONTROLE DE ACESSO
```

```
-- ============================================================================


CREATE TABLE nivel_acesso
(
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    descricao     VARCHAR(30) NOT NULL,
    criado_em     TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMPTZ NULL,
    CONSTRAINT uq_nivel_acesso_descricao
        UNIQUE (descricao),
    CONSTRAINT chk_nivel_acesso_descricao
        CHECK (descricao IN ('ADMIN', 'GESTAO', 'SUPERVISOR', 'OPERADOR', 'PCP'))
);
COMMENT ON TABLE nivel_acesso IS 'Perfis de acesso: ADMIN, SUPERVISOR, PCP, GESTAO e OPERADOR (hierarquia definida no
SecurityConfig)';


CREATE TABLE setor
(
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome          VARCHAR(50) NOT NULL,
    criado_em     TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMPTZ NULL,
    CONSTRAINT uq_setor_nome
        UNIQUE (nome)
);
COMMENT ON TABLE setor IS 'Setores de produção (ex: Corte, Dobra, Pintura)';


CREATE TABLE usuario
(
    id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome              VARCHAR(50)  NOT NULL,
    login             VARCHAR(20)  NOT NULL,
    senha             VARCHAR(255) NOT NULL,
    nivel_acesso_id   BIGINT       NOT NULL,
    setor_id          BIGINT       NULL,
    tentativas_falhas INT          NOT NULL DEFAULT 0,
    bloqueado_ate     TIMESTAMPTZ  NULL,
    criado_em         TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em     TIMESTAMPTZ  NULL,
    CONSTRAINT uq_usuario_login
        UNIQUE (login),
    CONSTRAINT fk_usuario_nivel_acesso
        FOREIGN KEY (nivel_acesso_id) REFERENCES nivel_acesso (id),
    CONSTRAINT fk_usuario_setor
        FOREIGN KEY (setor_id) REFERENCES setor (id) ON DELETE SET NULL,
    CONSTRAINT chk_usuario_tentativas_falhas
        CHECK (tentativas_falhas >= 0)
);
COMMENT ON TABLE usuario IS 'Usuários do sistema (login). Quem executa a produção é o operador, não o usuário';
COMMENT ON COLUMN usuario.tentativas_falhas IS 'Falhas consecutivas de login desde o último sucesso ou bloqueio';
COMMENT ON COLUMN usuario.bloqueado_ate IS 'Login bloqueado temporariamente até este instante (NULL = sem bloqueio)';


CREATE TABLE operador
(
    id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome            VARCHAR(100) NOT NULL,
    funcao          VARCHAR(30)  NULL,
    setor_padrao_id BIGINT       NULL,
    criado_em       TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em   TIMESTAMPTZ  NULL,
    CONSTRAINT uq_operador_nome
        UNIQUE (nome),
    CONSTRAINT fk_operador_setor_padrao
        FOREIGN KEY (setor_padrao_id) REFERENCES setor (id) ON DELETE SET NULL
);
COMMENT ON TABLE operador IS 'Operadores de produção, escolhidos nos formulários de lançamento';
COMMENT ON COLUMN operador.setor_padrao_id IS 'Setor de atuação (NULL = volante)';


-- ============================================================================
-- 2. ESTRUTURA DE PRODUÇÃO
-- ============================================================================


CREATE TABLE maquina
(
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome          VARCHAR(50) NOT NULL,
    setor_id      BIGINT      NOT NULL,
    criado_em     TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMPTZ NULL,
    CONSTRAINT uq_maquina_nome
        UNIQUE (nome),
    CONSTRAINT fk_maquina_setor
        FOREIGN KEY (setor_id) REFERENCES setor (id) ON DELETE RESTRICT
);
```

```
COMMENT ON TABLE maquina IS 'Máquinas instaladas em cada setor de produção';


CREATE TABLE etapa_setor
(
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome          VARCHAR(50) NOT NULL,
    ordem         INT         NOT NULL,
    setor_id      BIGINT      NOT NULL,
    criado_em     TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMPTZ NULL,
    CONSTRAINT fk_etapa_setor_setor
        FOREIGN KEY (setor_id) REFERENCES setor (id) ON DELETE CASCADE,
    CONSTRAINT chk_etapa_setor_ordem
        CHECK (ordem > 0),
    -- DEFERRABLE: a unicidade só é verificada no commit. Permite renumerar e
    -- remover etapas na mesma transação sem violação intermediária.
    CONSTRAINT uq_etapa_setor_ordem
        UNIQUE (setor_id, ordem) DEFERRABLE INITIALLY DEFERRED
);
COMMENT ON TABLE etapa_setor IS 'Etapas sequenciais dentro de cada setor';


CREATE TABLE roteiro
(
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    codigo        VARCHAR(30)  NOT NULL UNIQUE,
    descricao     VARCHAR(255) NULL,
    criado_em     TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMPTZ  NULL
);
COMMENT ON TABLE roteiro IS 'Roteiro padrão de produção, reutilizável por vários produtos';


CREATE TABLE produto
(
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome          VARCHAR(100) NOT NULL,
    codigo        VARCHAR(30)  NOT NULL UNIQUE,
    descricao     VARCHAR(255) NULL,
    roteiro_id    BIGINT       NULL,
    criado_em     TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMPTZ  NULL,
    CONSTRAINT fk_produto_roteiro
        FOREIGN KEY (roteiro_id) REFERENCES roteiro (id) ON DELETE RESTRICT
);
COMMENT ON TABLE produto IS 'Catálogo de produtos fabricados';


CREATE TABLE roteiro_passo
(
    id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    sequencia      INT    NOT NULL,
    roteiro_id     BIGINT NOT NULL,
    setor_id       BIGINT NOT NULL,
    etapa_setor_id BIGINT NULL,
    CONSTRAINT fk_passo_roteiro
        FOREIGN KEY (roteiro_id) REFERENCES roteiro (id) ON DELETE CASCADE,
    CONSTRAINT fk_passo_setor
        FOREIGN KEY (setor_id) REFERENCES setor (id) ON DELETE RESTRICT,
    CONSTRAINT fk_passo_etapa
        FOREIGN KEY (etapa_setor_id) REFERENCES etapa_setor (id) ON DELETE SET NULL,
    CONSTRAINT uq_passo_sequencia
        UNIQUE (roteiro_id, sequencia)
);
COMMENT ON TABLE roteiro_passo IS 'Passos (setor/etapa) de um roteiro, em ordem de sequência';


-- ================================================================================
-- 3. ORDENS DE PRODUÇÃO
-- ================================================================================


CREATE SEQUENCE ordem_producao_numero_seq START WITH 1 INCREMENT BY 1;


CREATE TABLE ordem_producao
(
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    numero        VARCHAR(30)  NOT NULL,
    quantidade    INT          NOT NULL,
    status        VARCHAR(25)  NOT NULL,
    produto_id    BIGINT       NOT NULL,
    criado_por_id BIGINT       NOT NULL,
    observacao    VARCHAR(500) NULL,
    data_criacao  TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    criado_em     TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMPTZ  NULL,
    CONSTRAINT uq_ordem_producao_numero
        UNIQUE (numero),
```

```
    CONSTRAINT fk_ordem_produto
        FOREIGN KEY (produto_id) REFERENCES produto (id) ON DELETE RESTRICT,
    CONSTRAINT fk_ordem_criado_por
        FOREIGN KEY (criado_por_id) REFERENCES usuario (id) ON DELETE RESTRICT,
    CONSTRAINT chk_ordem_quantidade
        CHECK (quantidade > 0),
    CONSTRAINT chk_ordem_status
        CHECK (status IN ('DISPONIVEL', 'LIBERADA', 'EM_ANDAMENTO', 'CONCLUIDA'))
);
COMMENT ON TABLE ordem_producao IS 'Ordens de produção abertas pelo PCP';


CREATE TABLE ordem_setor
(
    id               BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    sequencia        INT         NOT NULL,
    qtd_recebida     INT         NOT NULL DEFAULT 0,
    qtd_produzida    INT         NOT NULL DEFAULT 0,
    qtd_pendente     INT         NOT NULL DEFAULT 0,
    status           VARCHAR(25) NOT NULL,
    variante         VARCHAR(50) NULL,
    origem_refugo    BOOLEAN     NOT NULL DEFAULT FALSE,
    inicio           TIMESTAMPTZ NULL,
    fim              TIMESTAMPTZ NULL,
    versao           BIGINT      NOT NULL DEFAULT 0,
    ordem_producao_id BIGINT     NOT NULL,
    setor_id         BIGINT      NOT NULL,
    criado_em        TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em    TIMESTAMPTZ NULL,
    CONSTRAINT fk_ordem_setor_ordem
        FOREIGN KEY (ordem_producao_id) REFERENCES ordem_producao (id) ON DELETE CASCADE,
    CONSTRAINT fk_ordem_setor_setor
        FOREIGN KEY (setor_id) REFERENCES setor (id) ON DELETE RESTRICT,
    CONSTRAINT chk_ordem_setor_quantidades
        CHECK (qtd_recebida >= 0 AND qtd_produzida >= 0 AND qtd_pendente >= 0),
    CONSTRAINT chk_ordem_setor_status
        CHECK (status IN ('DISPONIVEL', 'LIBERADA', 'EM_ANDAMENTO', 'CONCLUIDA'))
);
COMMENT ON TABLE ordem_setor IS 'Linha de andamento da ordem em um setor (uma por sequência e variante, mais as de
refugo)';
COMMENT ON COLUMN ordem_setor.variante IS 'Variante/cor criada pela divisão da etapa (NULL = sem divisão)';
COMMENT ON COLUMN ordem_setor.origem_refugo IS 'TRUE = linha criada por um refugo para retrabalho';
COMMENT ON COLUMN ordem_setor.versao IS 'Controle de concorrência otimista (@Version)';


CREATE TABLE lancamento
(
    id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    qtd_produzida  INT          NOT NULL,
    observacao     VARCHAR(255) NULL,
    data_hora      TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ordem_setor_id BIGINT       NOT NULL,
    maquina_id     BIGINT       NOT NULL,
    operador_id    BIGINT       NOT NULL,
    etapa_setor_id BIGINT       NULL,
    CONSTRAINT fk_lancamento_ordem_setor
        FOREIGN KEY (ordem_setor_id) REFERENCES ordem_setor (id) ON DELETE RESTRICT,
    CONSTRAINT fk_lancamento_maquina
        FOREIGN KEY (maquina_id) REFERENCES maquina (id) ON DELETE RESTRICT,
    CONSTRAINT fk_lancamento_operador
        FOREIGN KEY (operador_id) REFERENCES operador (id) ON DELETE RESTRICT,
    CONSTRAINT fk_lancamento_etapa
        FOREIGN KEY (etapa_setor_id) REFERENCES etapa_setor (id) ON DELETE SET NULL,
    CONSTRAINT chk_lancamento_quantidade
        CHECK (qtd_produzida > 0)
);
COMMENT ON TABLE lancamento IS 'Registro de produção realizada por operador em máquina';


-- ============================================================================
-- 4. QUALIDADE E CONTROLE
-- ============================================================================


CREATE TABLE refugo
(
    id                        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    qtd_refugo                INT          NOT NULL,
    motivo                    VARCHAR(255) NOT NULL,
    destino                   VARCHAR(30)  NOT NULL,
    variante                  VARCHAR(50)  NULL,
    data_hora                 TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    sequencia_origem          INT          NOT NULL,
    ordem_setor_id            BIGINT       NOT NULL,
    setor_origem_id           BIGINT       NOT NULL,
    operador_id               BIGINT       NOT NULL,
    ordem_setor_retrabalho_id BIGINT       NULL,
    criado_em                 TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em             TIMESTAMPTZ  NULL,
    CONSTRAINT fk_refugo_ordem_setor
```

```
        FOREIGN KEY (ordem_setor_id) REFERENCES ordem_setor (id) ON DELETE RESTRICT,
    CONSTRAINT fk_refugo_setor_origem
        FOREIGN KEY (setor_origem_id) REFERENCES setor (id) ON DELETE RESTRICT,
    CONSTRAINT fk_refugo_operador
        FOREIGN KEY (operador_id) REFERENCES operador (id) ON DELETE RESTRICT,
    CONSTRAINT fk_refugo_ordem_setor_retrabalho
        FOREIGN KEY (ordem_setor_retrabalho_id) REFERENCES ordem_setor (id) ON DELETE SET NULL,
    CONSTRAINT chk_refugo_destino
        CHECK (destino IN ('RETRABALHO', 'DESCARTE')),
    CONSTRAINT chk_refugo_quantidade
        CHECK (qtd_refugo > 0)
);
COMMENT ON TABLE refugo IS 'Registro de peças refugadas com motivo e destino';
COMMENT ON COLUMN refugo.ordem_setor_id IS 'Linha onde o refugo foi detectado';
COMMENT ON COLUMN refugo.ordem_setor_retrabalho_id IS 'Linha criada no setor de origem para o retrabalho (NULL no
descarte)';


CREATE TABLE retrabalho
(
    id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    qtd_refeita   INT          NOT NULL,
    observacao    VARCHAR(255) NULL,
    data_hora     TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    refugo_id     BIGINT       NOT NULL,
    maquina_id    BIGINT       NULL,
    operador_id   BIGINT       NOT NULL,
    criado_em     TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em TIMESTAMPTZ  NULL,
    CONSTRAINT fk_retrabalho_refugo
        FOREIGN KEY (refugo_id) REFERENCES refugo (id) ON DELETE CASCADE,
    CONSTRAINT fk_retrabalho_maquina
        FOREIGN KEY (maquina_id) REFERENCES maquina (id) ON DELETE RESTRICT,
    CONSTRAINT fk_retrabalho_operador
        FOREIGN KEY (operador_id) REFERENCES operador (id) ON DELETE RESTRICT,
    CONSTRAINT chk_retrabalho_quantidade
        CHECK (qtd_refeita > 0)
);
COMMENT ON TABLE retrabalho IS 'Retrabalho de peças refugadas';


-- ============================================================================
-- 5. CONTROLE DE VIDROS (ledger de movimentações)
-- ============================================================================


CREATE TABLE chapa_vidro
(
    id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tipo_vidro     VARCHAR(20)  NOT NULL,
    descricao      VARCHAR(100) NOT NULL,
    estoque_minimo INT          NOT NULL DEFAULT 5,
    criado_em      TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em  TIMESTAMPTZ  NULL,
    CONSTRAINT uq_chapa_vidro_tipo
        UNIQUE (tipo_vidro),
    CONSTRAINT chk_chapa_vidro_estoque_minimo
        CHECK (estoque_minimo >= 0)
);
COMMENT ON TABLE chapa_vidro IS 'Catálogo de chapas/tipos de vidro';


CREATE TABLE movimentacao_vidro
(
    id                BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    chapa_vidro_id    BIGINT       NOT NULL,
    tipo_movimentacao VARCHAR(20)  NOT NULL,
    quantidade        INT          NOT NULL,
    observacao        VARCHAR(255) NULL,
    data_hora         TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    operador_id       BIGINT       NOT NULL,
    criado_em         TIMESTAMPTZ  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atualizado_em     TIMESTAMPTZ  NULL,
    CONSTRAINT fk_movimentacao_chapa_vidro
        FOREIGN KEY (chapa_vidro_id) REFERENCES chapa_vidro (id) ON DELETE RESTRICT,
    CONSTRAINT fk_movimentacao_operador
        FOREIGN KEY (operador_id) REFERENCES operador (id) ON DELETE RESTRICT,
    CONSTRAINT chk_movimentacao_quantidade
        CHECK (quantidade > 0),
    CONSTRAINT chk_movimentacao_tipo
        CHECK (tipo_movimentacao IN ('ENTRADA', 'SAIDA'))
);
COMMENT ON TABLE movimentacao_vidro IS 'Ledger de entradas e saídas de vidro; o saldo é a soma das movimentações';


-- ============================================================================
-- 6. DADOS INICIAIS
-- ============================================================================
-- A ordem dos INSERTs define os ids (ADMIN=1 ... PCP=5) e preserva os ids de
-- bancos criados pelo histórico antigo de migrations.
```

```
INSERT INTO nivel_acesso (descricao)
VALUES ('ADMIN'),
       ('GESTAO'),
       ('SUPERVISOR'),
       ('OPERADOR'),
       ('PCP');
```

V2__create_triggers.sql

```
-- ================================================================================
-- V2 — FUNÇÕES E TRIGGERS
-- ================================================================================
-- Mantém atualizado_em em todas as tabelas auditadas. Não existem triggers em
-- lancamento e roteiro_produto porque essas tabelas não têm a coluna.
-- ================================================================================


-- ================================================================================
-- 1. FUNÇÕES
-- ================================================================================

CREATE OR REPLACE FUNCTION update_atualizado_em_column()
    RETURNS TRIGGER AS
$$
BEGIN
    NEW.atualizado_em = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

COMMENT ON FUNCTION update_atualizado_em_column() IS 'Atualiza atualizado_em antes de cada UPDATE em tabelas auditadas';


-- ================================================================================
-- 2. TRIGGERS DE AUDITORIA (atualizado_em)
-- ================================================================================
-- A cláusula WHEN evita alterar atualizado_em em UPDATEs que não mudam nenhum valor.

CREATE TRIGGER trg_nivel_acesso_atualizado_em
    BEFORE UPDATE ON nivel_acesso
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();

CREATE TRIGGER trg_setor_atualizado_em
    BEFORE UPDATE ON setor
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();

CREATE TRIGGER trg_usuario_atualizado_em
    BEFORE UPDATE ON usuario
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();

CREATE TRIGGER trg_operador_atualizado_em
    BEFORE UPDATE ON operador
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();

CREATE TRIGGER trg_maquina_atualizado_em
    BEFORE UPDATE ON maquina
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();

CREATE TRIGGER trg_etapa_setor_atualizado_em
    BEFORE UPDATE ON etapa_setor
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();

CREATE TRIGGER trg_produto_atualizado_em
    BEFORE UPDATE ON produto
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();

CREATE TRIGGER trg_roteiro_atualizado_em
    BEFORE UPDATE ON roteiro FOR EACH ROW
EXECUTE FUNCTION update_atualizado_em_column();
```

```
CREATE TRIGGER trg_ordem_producao_atualizado_em
    BEFORE UPDATE ON ordem_producao
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();


CREATE TRIGGER trg_ordem_setor_atualizado_em
    BEFORE UPDATE ON ordem_setor
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();


CREATE TRIGGER trg_refugo_atualizado_em
    BEFORE UPDATE ON refugo
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();


CREATE TRIGGER trg_retrabalho_atualizado_em
    BEFORE UPDATE ON retrabalho
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();


CREATE TRIGGER trg_chapa_vidro_atualizado_em
    BEFORE UPDATE ON chapa_vidro
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();


CREATE TRIGGER trg_movimentacao_vidro_atualizado_em
    BEFORE UPDATE ON movimentacao_vidro
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();
```

V3__create_indexes.sql

```
-- ============================================================================
-- V3 — ÍNDICES DE DESEMPENHO
-- ============================================================================
-- Regras:
--   * PostgreSQL NÃO cria índice automático para colunas de chave estrangeira:
--     todas as FKs têm índice (acelera JOINs e DELETEs na tabela referenciada).
--   * Constraints UNIQUE (V1) já criam seus próprios índices; não são repetidos aqui.
--   * IF NOT EXISTS torna a migration segura para reexecução.
-- ============================================================================


-- USUÁRIO E OPERADOR
CREATE INDEX IF NOT EXISTS idx_usuario_nivel_acesso ON usuario (nivel_acesso_id);
CREATE INDEX IF NOT EXISTS idx_usuario_setor ON usuario (setor_id);
CREATE INDEX IF NOT EXISTS idx_operador_setor_padrao ON operador (setor_padrao_id);


-- MÁQUINA
CREATE INDEX IF NOT EXISTS idx_maquina_setor ON maquina (setor_id);


-- ROTEIRO (roteiro_id é coberto por uq_passo_sequencia; etapa_setor.setor_id por uq_etapa_setor_ordem)
CREATE INDEX IF NOT EXISTS idx_produto_roteiro ON produto (roteiro_id);
CREATE INDEX IF NOT EXISTS idx_roteiro_passo_setor ON roteiro_passo (setor_id);
CREATE INDEX IF NOT EXISTS idx_roteiro_passo_etapa ON roteiro_passo (etapa_setor_id);


-- ORDEM DE PRODUÇÃO
-- (produto_id, status) atende a FK e a consulta "existe ordem aberta deste produto".
CREATE INDEX IF NOT EXISTS idx_ordem_produto_status ON ordem_producao (produto_id, status);
CREATE INDEX IF NOT EXISTS idx_ordem_status ON ordem_producao (status);
CREATE INDEX IF NOT EXISTS idx_ordem_criado_por ON ordem_producao (criado_por_id);
CREATE INDEX IF NOT EXISTS idx_ordem_data_criacao ON ordem_producao (data_criacao DESC);


-- ORDEM SETOR
CREATE INDEX IF NOT EXISTS idx_ordem_setor_ordem_producao ON ordem_setor (ordem_producao_id);
CREATE INDEX IF NOT EXISTS idx_ordem_setor_setor ON ordem_setor (setor_id);
CREATE INDEX IF NOT EXISTS idx_ordem_setor_status ON ordem_setor (status);


-- Regra de negócio: no máximo UMA linha normal por ordem + sequência + variante.
-- Linhas criadas por refugo (origem_refugo = TRUE) ficam fora da regra.
CREATE UNIQUE INDEX IF NOT EXISTS uq_ordem_setor_linha_normal
    ON ordem_setor (ordem_producao_id, sequencia, (COALESCE(variante, '')))
    WHERE origem_refugo = FALSE;


-- LANÇAMENTO
CREATE INDEX IF NOT EXISTS idx_lancamento_ordem_setor ON lancamento (ordem_setor_id);
CREATE INDEX IF NOT EXISTS idx_lancamento_maquina ON lancamento (maquina_id);
CREATE INDEX IF NOT EXISTS idx_lancamento_operador ON lancamento (operador_id);
CREATE INDEX IF NOT EXISTS idx_lancamento_etapa ON lancamento (etapa_setor_id);
```

```
CREATE INDEX IF NOT EXISTS idx_lancamento_data_hora ON lancamento (data_hora DESC);


-- REFUGO E RETRABALHO
CREATE INDEX IF NOT EXISTS idx_refugo_ordem_setor ON refugo (ordem_setor_id);
CREATE INDEX IF NOT EXISTS idx_refugo_setor_origem ON refugo (setor_origem_id);
CREATE INDEX IF NOT EXISTS idx_refugo_operador ON refugo (operador_id);
CREATE INDEX IF NOT EXISTS idx_refugo_ordem_setor_retrabalho ON refugo (ordem_setor_retrabalho_id);
CREATE INDEX IF NOT EXISTS idx_refugo_data_hora ON refugo (data_hora DESC);
CREATE INDEX IF NOT EXISTS idx_retrabalho_refugo ON retrabalho (refugo_id);
CREATE INDEX IF NOT EXISTS idx_retrabalho_maquina ON retrabalho (maquina_id);
CREATE INDEX IF NOT EXISTS idx_retrabalho_operador ON retrabalho (operador_id);


-- VIDRO
-- (chapa, tipo) atende a FK e a soma de saldo por chapa e tipo de movimentação.
CREATE INDEX IF NOT EXISTS idx_movimentacao_vidro_chapa_tipo ON movimentacao_vidro (chapa_vidro_id, tipo_movimentacao);
CREATE INDEX IF NOT EXISTS idx_movimentacao_vidro_operador ON movimentacao_vidro (operador_id);
CREATE INDEX IF NOT EXISTS idx_movimentacao_vidro_data ON movimentacao_vidro (data_hora DESC);
```

V4__ordem_producao_roteiro.sql

```
-- ============================================================================
-- V4 — ROTEIRO GRAVADO NA ORDEM DE PRODUÇÃO
-- ============================================================================
-- A ordem passa a guardar o roteiro com que foi criada. Assim, trocar o roteiro
-- de um produto (ou criar uma nova versão por duplicação) não reinterpreta as
-- ordens abertas nem o histórico das concluídas.
-- ============================================================================


ALTER TABLE ordem_producao
    ADD COLUMN roteiro_id BIGINT NULL;


-- Ordens já existentes recebem o roteiro atual do produto.
UPDATE ordem_producao o
SET roteiro_id = p.roteiro_id
FROM produto p
WHERE p.id = o.produto_id;


ALTER TABLE ordem_producao
    ADD CONSTRAINT fk_ordem_roteiro
        FOREIGN KEY (roteiro_id) REFERENCES roteiro (id) ON DELETE RESTRICT;


CREATE INDEX IF NOT EXISTS idx_ordem_roteiro ON ordem_producao (roteiro_id);


-- NOT NULL somente quando o preenchimento acima cobriu todas as linhas.
-- (Ordem antiga cujo produto já estava sem roteiro deve ser tratada manualmente.)
DO
$$
    BEGIN
        IF NOT EXISTS (SELECT 1 FROM ordem_producao WHERE roteiro_id IS NULL) THEN
            ALTER TABLE ordem_producao
                ALTER COLUMN roteiro_id SET NOT NULL;
        END IF;
    END
$$;


COMMENT ON COLUMN ordem_producao.roteiro_id IS 'Roteiro usado na criação da ordem; define as etapas e a sequência dela';
```

V5__indices_escala.sql

```
-- ============================================================================
-- V5 — ÍNDICES PARA ESCALA
-- ============================================================================


CREATE INDEX IF NOT EXISTS idx_ordem_setor_setor_status_criado
    ON ordem_setor (setor_id, status, criado_em DESC);


DROP INDEX IF EXISTS idx_ordem_setor_setor;


DO
$$
    BEGIN
        CREATE EXTENSION IF NOT EXISTS pg_trgm;
        EXECUTE 'CREATE INDEX IF NOT EXISTS idx_produto_nome_trgm
                 ON produto USING gin (lower(nome) gin_trgm_ops)';
        EXECUTE 'CREATE INDEX IF NOT EXISTS idx_produto_codigo_trgm
                 ON produto USING gin (lower(codigo) gin_trgm_ops)';
```

```
        EXECUTE 'CREATE INDEX IF NOT EXISTS idx_ordem_numero_trgm
                 ON ordem_producao USING gin (lower(numero) gin_trgm_ops)';
    EXCEPTION
        WHEN OTHERS THEN
            RAISE NOTICE 'pg_trgm indisponível; índices de busca por trecho não criados: %', SQLERRM;
    END
$$;
```

V6__produto_nome_opcional.sql

```
-- ================================================================================
-- V6 — NOME DO PRODUTO OPCIONAL
-- ================================================================================
-- O código é a identificação principal do produto; o nome passa a ser opcional.
-- ================================================================================

ALTER TABLE produto
    ALTER COLUMN nome DROP NOT NULL;

COMMENT ON COLUMN produto.nome IS 'Nome opcional; o código é a identificação principal do produto';
```

4.6 Inserções no banco de dados

Inserções dos dados de demonstração usados neste documento, no tema de uma metalúrgica de luminárias. Os níveis de acesso são inseridos pela própria migração V1. As tabelas de cadastro foram populadas pelos comandos abaixo; as tabelas de movimento (ordens, etapas, lançamentos, refugos, retrabalhos e movimentações de vidro) foram preenchidas pelo próprio sistema, por meio das suas telas, e estão representadas como INSERT equivalentes.

```
-- Tabela setor
INSERT INTO setor (nome) VALUES ('CORTE');
INSERT INTO setor (nome) VALUES ('DOBRA');
INSERT INTO setor (nome) VALUES ('SOLDA');
INSERT INTO setor (nome) VALUES ('PINTURA');
INSERT INTO setor (nome) VALUES ('MONTAGEM');


-- Tabela etapa_setor
INSERT INTO etapa_setor (nome, ordem, setor_id) VALUES ('1ª DOBRA', 1, (SELECT id FROM setor WHERE nome =
'DOBRA'));
INSERT INTO etapa_setor (nome, ordem, setor_id) VALUES ('2ª DOBRA', 2, (SELECT id FROM setor WHERE nome =
'DOBRA'));
INSERT INTO etapa_setor (nome, ordem, setor_id) VALUES ('PRÉ-TRATAMENTO', 1, (SELECT id FROM setor WHERE nome =
'PINTURA'));
INSERT INTO etapa_setor (nome, ordem, setor_id) VALUES ('PINTURA EPÓXI', 2, (SELECT id FROM setor WHERE nome =
'PINTURA'));


-- Tabela maquina
INSERT INTO maquina (nome, setor_id) VALUES ('GUILHOTINA 01', (SELECT id FROM setor WHERE nome = 'CORTE'));
INSERT INTO maquina (nome, setor_id) VALUES ('LASER FIBRA 01', (SELECT id FROM setor WHERE nome = 'CORTE'));
INSERT INTO maquina (nome, setor_id) VALUES ('DOBRADEIRA 01', (SELECT id FROM setor WHERE nome = 'DOBRA'));
INSERT INTO maquina (nome, setor_id) VALUES ('DOBRADEIRA 02', (SELECT id FROM setor WHERE nome = 'DOBRA'));
INSERT INTO maquina (nome, setor_id) VALUES ('SOLDA MIG 01', (SELECT id FROM setor WHERE nome = 'SOLDA'));
INSERT INTO maquina (nome, setor_id) VALUES ('CABINE PINTURA 01', (SELECT id FROM setor WHERE nome = 'PINTURA'));
INSERT INTO maquina (nome, setor_id) VALUES ('BANCADA MONTAGEM 01', (SELECT id FROM setor WHERE nome =
'MONTAGEM'));


-- Tabela operador
INSERT INTO operador (nome, funcao, setor_padrao_id) VALUES ('CARLOS SILVA', 'CORTADOR', (SELECT id FROM setor
WHERE nome = 'CORTE'));
INSERT INTO operador (nome, funcao, setor_padrao_id) VALUES ('MARCOS OLIVEIRA', 'DOBRADOR', (SELECT id FROM setor
WHERE nome = 'DOBRA'));
INSERT INTO operador (nome, funcao, setor_padrao_id) VALUES ('ANA SOUZA', 'SOLDADORA', (SELECT id FROM setor WHERE
nome = 'SOLDA'));
INSERT INTO operador (nome, funcao, setor_padrao_id) VALUES ('PAULO LIMA', 'PINTOR', (SELECT id FROM setor WHERE
nome = 'PINTURA'));
INSERT INTO operador (nome, funcao, setor_padrao_id) VALUES ('JULIANA COSTA', 'MONTADORA', (SELECT id FROM setor
WHERE nome = 'MONTAGEM'));
INSERT INTO operador (nome, funcao, setor_padrao_id) VALUES ('RAFAEL SANTOS', 'VOLANTE', NULL);


-- Tabela roteiro
INSERT INTO roteiro (codigo, descricao) VALUES ('RT-ARANDELA', 'ARANDELAS EM AÇO PINTADO');
INSERT INTO roteiro (codigo, descricao) VALUES ('RT-PENDENTE', 'PENDENTES COM CÚPULA DOBRADA');
```

```
INSERT INTO roteiro (codigo, descricao) VALUES ('RT-PLAFON', 'PLAFONS DE SOBREPOR');


-- Tabela roteiro_passo
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (1, (SELECT id FROM roteiro
WHERE codigo = 'RT-ARANDELA'), (SELECT id FROM setor WHERE nome = 'CORTE'), NULL);
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (2, (SELECT id FROM roteiro
WHERE codigo = 'RT-ARANDELA'), (SELECT id FROM setor WHERE nome = 'DOBRA'), (SELECT id FROM etapa_setor WHERE nome
= '1ª DOBRA'));
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (3, (SELECT id FROM roteiro
WHERE codigo = 'RT-ARANDELA'), (SELECT id FROM setor WHERE nome = 'SOLDA'), NULL);
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (4, (SELECT id FROM roteiro
WHERE codigo = 'RT-ARANDELA'), (SELECT id FROM setor WHERE nome = 'PINTURA'), (SELECT id FROM etapa_setor WHERE
nome = 'PINTURA EPÓXI'));
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (5, (SELECT id FROM roteiro
WHERE codigo = 'RT-ARANDELA'), (SELECT id FROM setor WHERE nome = 'MONTAGEM'), NULL);
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (1, (SELECT id FROM roteiro
WHERE codigo = 'RT-PENDENTE'), (SELECT id FROM setor WHERE nome = 'CORTE'), NULL);
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (2, (SELECT id FROM roteiro
WHERE codigo = 'RT-PENDENTE'), (SELECT id FROM setor WHERE nome = 'DOBRA'), (SELECT id FROM etapa_setor WHERE nome
= '1ª DOBRA'));
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (3, (SELECT id FROM roteiro
WHERE codigo = 'RT-PENDENTE'), (SELECT id FROM setor WHERE nome = 'DOBRA'), (SELECT id FROM etapa_setor WHERE nome
= '2ª DOBRA'));
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (4, (SELECT id FROM roteiro
WHERE codigo = 'RT-PENDENTE'), (SELECT id FROM setor WHERE nome = 'PINTURA'), (SELECT id FROM etapa_setor WHERE
nome = 'PINTURA EPÓXI'));
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (5, (SELECT id FROM roteiro
WHERE codigo = 'RT-PENDENTE'), (SELECT id FROM setor WHERE nome = 'MONTAGEM'), NULL);
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (1, (SELECT id FROM roteiro
WHERE codigo = 'RT-PLAFON'), (SELECT id FROM setor WHERE nome = 'CORTE'), NULL);
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (2, (SELECT id FROM roteiro
WHERE codigo = 'RT-PLAFON'), (SELECT id FROM setor WHERE nome = 'PINTURA'), (SELECT id FROM etapa_setor WHERE nome
= 'PINTURA EPÓXI'));
INSERT INTO roteiro_passo (sequencia, roteiro_id, setor_id, etapa_setor_id) VALUES (3, (SELECT id FROM roteiro
WHERE codigo = 'RT-PLAFON'), (SELECT id FROM setor WHERE nome = 'MONTAGEM'), NULL);


-- Tabela produto
INSERT INTO produto (nome, codigo, descricao, roteiro_id) VALUES ('ARANDELA TUBO 10CM', 'ARD-1010', 'ARANDELA DE
PAREDE EM TUBO, PINTURA PRETA FOSCA', (SELECT id FROM roteiro WHERE codigo = 'RT-ARANDELA'));
INSERT INTO produto (nome, codigo, descricao, roteiro_id) VALUES ('ARANDELA MEIA LUA', 'ARD-1020', 'ARANDELA MEIA
LUA PARA ÁREA EXTERNA', (SELECT id FROM roteiro WHERE codigo = 'RT-ARANDELA'));
INSERT INTO produto (nome, codigo, descricao, roteiro_id) VALUES ('PENDENTE CONE 30CM', 'PEN-2030', 'PENDENTE
CÔNICO COM CÚPULA DOBRADA', (SELECT id FROM roteiro WHERE codigo = 'RT-PENDENTE'));
INSERT INTO produto (nome, codigo, descricao, roteiro_id) VALUES ('PENDENTE DOMUS 40CM', 'PEN-2040', 'PENDENTE
DOMUS COM DIFUSOR DE VIDRO', (SELECT id FROM roteiro WHERE codigo = 'RT-PENDENTE'));
INSERT INTO produto (nome, codigo, descricao, roteiro_id) VALUES ('PLAFON QUADRADO 25CM', 'PLF-3025', 'PLAFON DE
SOBREPOR COM VIDRO JATEADO', (SELECT id FROM roteiro WHERE codigo = 'RT-PLAFON'));


-- Tabela chapa_vidro
INSERT INTO chapa_vidro (tipo_vidro, descricao, estoque_minimo) VALUES ('INCOLOR', 'VIDRO INCOLOR 4MM', 10);
INSERT INTO chapa_vidro (tipo_vidro, descricao, estoque_minimo) VALUES ('JATEADO', 'VIDRO JATEADO 4MM', 8);
INSERT INTO chapa_vidro (tipo_vidro, descricao, estoque_minimo) VALUES ('REFLEXIVEL', 'VIDRO REFLEXIVO 4MM', 5);
```

-- Tabela usuario

```
INSERT INTO usuario (nome, login, senha, nivel_acesso_id) VALUES ('ADMINISTRADOR', 'admin', '<hash BCrypt>',
(SELECT id FROM nivel_acesso WHERE descricao = 'ADMIN'));
INSERT INTO usuario (nome, login, senha, nivel_acesso_id) VALUES ('PLANEJAMENTO PCP', 'pcp', '<hash BCrypt>',
(SELECT id FROM nivel_acesso WHERE descricao = 'PCP'));
INSERT INTO usuario (nome, login, senha, nivel_acesso_id) VALUES ('GESTOR PRODUCAO', 'gestao', '<hash BCrypt>',
(SELECT id FROM nivel_acesso WHERE descricao = 'GESTAO'));
INSERT INTO usuario (nome, login, senha, nivel_acesso_id, setor_id) VALUES ('OPERADOR DOBRA', 'op.dobra', '<hash
BCrypt>', (SELECT id FROM nivel_acesso WHERE descricao = 'OPERADOR'), (SELECT id FROM setor WHERE nome =
'DOBRA'));
```

A senha nunca é gravada em texto puro: o campo senha recebe o hash BCrypt gerado pela aplicação.

-- Tabela ordem_producao

```
INSERT INTO ordem_producao (numero, quantidade, status, produto_id, roteiro_id, criado_por_id, observacao,
data_criacao) VALUES ('OP-0001', 100, 'DISPONIVEL', 1, 1, 1, 'PEDIDO LOJA CENTRO', '2025-11-03');
INSERT INTO ordem_producao (numero, quantidade, status, produto_id, roteiro_id, criado_por_id, observacao,
data_criacao) VALUES ('OP-0002', 50, 'DISPONIVEL', 3, 2, 1, 'CORES PRETO E BRANCO', '2025-11-04');
INSERT INTO ordem_producao (numero, quantidade, status, produto_id, roteiro_id, criado_por_id, observacao,
data_criacao) VALUES ('OP-0003', 80, 'DISPONIVEL', 5, 3, 1, NULL, '2025-11-05');
INSERT INTO ordem_producao (numero, quantidade, status, produto_id, roteiro_id, criado_por_id, observacao,
data_criacao) VALUES ('OP-0004', 30, 'DISPONIVEL', 2, 1, 1, 'AGUARDANDO LIBERAÇÃO', '2025-11-06');
INSERT INTO ordem_producao (numero, quantidade, status, produto_id, roteiro_id, criado_por_id, observacao,
```

```
data_criacao) VALUES ('OP-0005', 10, 'CONCLUIDA', 5, 3, 1, 'LOTE PILOTO', '2025-11-03');
```

-- Tabela ordem_setor

```
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (1, 1, 1, 100, 100, 0, 'CONCLUIDA', NULL, f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (2, 1, 1, 50, 50, 0, 'CONCLUIDA', NULL, f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (3, 1, 1, 80, 40, 40, 'EM_ANDAMENTO', NULL, f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (4, 1, 1, 30, 0, 30, 'DISPONIVEL', NULL, f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (5, 1, 1, 10, 10, 0, 'CONCLUIDA', NULL, f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (1, 2, 2, 97, 60, 37, 'EM_ANDAMENTO', NULL, f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (1, 3, 3, 60, 0, 60, 'DISPONIVEL', NULL, f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (1, 1, 1, 4, 4, 0, 'CONCLUIDA', NULL, t);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (2, 2, 2, 30, 30, 0, 'CONCLUIDA', 'PRETO', f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (2, 2, 2, 20, 0, 20, 'DISPONIVEL', 'BRANCO', f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (2, 2, 3, 30, 0, 30, 'DISPONIVEL', 'PRETO', f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (3, 4, 2, 40, 0, 40, 'DISPONIVEL', NULL, f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (5, 4, 2, 10, 10, 0, 'CONCLUIDA', NULL, f);
INSERT INTO ordem_setor (ordem_producao_id, setor_id, sequencia, qtd_recebida, qtd_produzida, qtd_pendente,
status, variante, origem_refugo) VALUES (5, 5, 3, 10, 10, 0, 'CONCLUIDA', NULL, f);
```

-- Tabela lancamento

```
INSERT INTO lancamento (ordem_setor_id, maquina_id, operador_id, etapa_setor_id, qtd_produzida, observacao) VALUES
(1, 1, 1, NULL, 100, NULL);
INSERT INTO lancamento (ordem_setor_id, maquina_id, operador_id, etapa_setor_id, qtd_produzida, observacao) VALUES
(6, 3, 2, 1, 60, 'TURNO MANHÃ');
INSERT INTO lancamento (ordem_setor_id, maquina_id, operador_id, etapa_setor_id, qtd_produzida, observacao) VALUES
(2, 2, 1, NULL, 50, NULL);
INSERT INTO lancamento (ordem_setor_id, maquina_id, operador_id, etapa_setor_id, qtd_produzida, observacao) VALUES
(10, 4, 2, 1, 30, NULL);
INSERT INTO lancamento (ordem_setor_id, maquina_id, operador_id, etapa_setor_id, qtd_produzida, observacao) VALUES
(3, 1, 1, NULL, 40, NULL);
INSERT INTO lancamento (ordem_setor_id, maquina_id, operador_id, etapa_setor_id, qtd_produzida, observacao) VALUES
(5, 2, 6, NULL, 10, NULL);
INSERT INTO lancamento (ordem_setor_id, maquina_id, operador_id, etapa_setor_id, qtd_produzida, observacao) VALUES
(14, 6, 4, 4, 10, NULL);
INSERT INTO lancamento (ordem_setor_id, maquina_id, operador_id, etapa_setor_id, qtd_produzida, observacao) VALUES
(15, 7, 5, NULL, 10, NULL);
```

-- Tabela refugo

```
INSERT INTO refugo (ordem_setor_id, setor_origem_id, operador_id, sequencia_origem, qtd_refugo, motivo, destino,
ordem_setor_retrabalho_id) VALUES (6, 2, 2, 2, 3, 'TRINCA NA DOBRA', 'DESCARTE', NULL);
INSERT INTO refugo (ordem_setor_id, setor_origem_id, operador_id, sequencia_origem, qtd_refugo, motivo, destino,
ordem_setor_retrabalho_id) VALUES (6, 1, 2, 1, 4, 'REBARBA NO CORTE', 'RETRABALHO', 8);
```

-- Tabela retrabalho

```
INSERT INTO retrabalho (refugo_id, maquina_id, operador_id, qtd_refeita, observacao) VALUES (2, 1, 1, 4, 'REBARBA
REMOVIDA');
```

-- Tabela movimentacao_vidro

```
INSERT INTO movimentacao_vidro (chapa_vidro_id, tipo_movimentacao, quantidade, operador_id, data_hora, observacao)
VALUES (1, 'ENTRADA', 40, 6, '2025-11-03 08:00', 'COMPRA FORNECEDOR');
INSERT INTO movimentacao_vidro (chapa_vidro_id, tipo_movimentacao, quantidade, operador_id, data_hora, observacao)
VALUES (2, 'ENTRADA', 25, 6, '2025-11-03 08:00', 'COMPRA FORNECEDOR');
INSERT INTO movimentacao_vidro (chapa_vidro_id, tipo_movimentacao, quantidade, operador_id, data_hora, observacao)
VALUES (3, 'ENTRADA', 12, 6, '2025-11-03 08:00', 'COMPRA FORNECEDOR');
INSERT INTO movimentacao_vidro (chapa_vidro_id, tipo_movimentacao, quantidade, operador_id, data_hora, observacao)
VALUES (2, 'SAIDA', 18, 5, '2025-11-05 14:30', 'CONSUMO MONTAGEM');
INSERT INTO movimentacao_vidro (chapa_vidro_id, tipo_movimentacao, quantidade, operador_id, data_hora, observacao)
```

```
VALUES (1, 'SAIDA', 6, 5, '2025-11-05 14:30', 'CONSUMO MONTAGEM');
INSERT INTO movimentacao_vidro (chapa_vidro_id, tipo_movimentacao, quantidade, operador_id, data_hora, observacao)
VALUES (3, 'SAIDA', 9, 5, '2025-11-05 14:30', 'CONSUMO MONTAGEM');
```

4.7 Views

O LumiFlow não utiliza views. As consultas que combinam várias tabelas são escritas em JPQL nos repositórios Spring Data JPA e executadas pela aplicação. Exemplos: a listagem de ordens por setor (OrdemSetorRepository.listarPorSetorIdEFiltros), que une ordem_setor, ordem_producao, produto e setor; o histórico de lançamentos (LancamentoRepository), que une lancamento, ordem_setor, maquina e operador; e o saldo de vidro (MovimentacaoVidroRepository.somarSaldosPorChapa), que agrega movimentacao_vidro por chapa.

4.8 Funções Escalares

O projeto não possui funções escalares. Os cálculos de apoio, como o número sequencial da ordem (OP-0001, gerado a partir da sequência ordem_producao_numero_seq), o progresso da ordem e o status consolidado, são feitos na camada de serviços (OrdemProducaoService e StatusOrdemResolver). A única função do banco é a função de trigger update_atualizado_em_column, apresentada na seção 4.11.

4.9 Funções do Tipo Table

O projeto não possui funções do tipo table. Os conjuntos de dados das telas, como o resumo por setor, os indicadores do dashboard e o histórico filtrado, são montados pelos serviços (DashboardService, OrdemSetorService e HistoricoLancamentosService) a partir de consultas paginadas dos repositórios.

4.10 Procedures

O projeto não possui stored procedures. As inserções e atualizações são feitas pela aplicação, dentro de transações (@Transactional), o que permite aplicar no mesmo lugar as validações de negócio, o controle de concorrência (coluna versao em ordem_setor e bloqueios pessimistas) e as mensagens de erro ao usuário. Exemplos: OrdemProducaoService.criar (abre a ordem e cria a primeira etapa) e LancamentoService.lancarProducao (registra o lançamento, atualiza as quantidades e encaminha a produção para a próxima etapa).

4.11 Triggers

O banco possui 14 triggers de auditoria, uma em cada tabela que tem a coluna atualizado_em. Antes de cada UPDATE, a trigger chama a função update_atualizado_em_column, que grava a data e a hora da alteração. A cláusula WHEN evita atualizar a data quando o UPDATE não muda nenhum valor. As tabelas lancamento e roteiro_passo não têm triggers porque não possuem essa coluna.

1. Função de trigger

```
CREATE OR REPLACE FUNCTION update_atualizado_em_column()
    RETURNS TRIGGER AS
$$
BEGIN
    NEW.atualizado_em = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

2. Triggers (mesmo padrão nas 14 tabelas)

```
CREATE TRIGGER trg_nivel_acesso_atualizado_em
```

```
    BEFORE UPDATE ON nivel_acesso
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();
CREATE TRIGGER trg_setor_atualizado_em
    BEFORE UPDATE ON setor
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();
CREATE TRIGGER trg_usuario_atualizado_em
    BEFORE UPDATE ON usuario
    FOR EACH ROW WHEN (OLD.* IS DISTINCT FROM NEW.*)
EXECUTE FUNCTION update_atualizado_em_column();
-- ... e as demais: operador, maquina, etapa_setor, produto, roteiro, ordem_producao,
--     ordem_setor, refugo, retrabalho, chapa_vidro e movimentacao_vidro.
```

3. Teste de execução

```
BEGIN;
SELECT nome, criado_em, atualizado_em FROM maquina WHERE nome = 'DOBRADEIRA 02';
--      nome      |      criado_em      | atualizado_em
-- DOBRADEIRA 02  | 2026-11-06 19:49:15 |   (nulo)

UPDATE maquina SET nome = 'DOBRADEIRA 02 CNC' WHERE nome = 'DOBRADEIRA 02';

SELECT nome, criado_em, atualizado_em FROM maquina WHERE nome = 'DOBRADEIRA 02 CNC';
--        nome        |      criado_em      |    atualizado_em
-- DOBRADEIRA 02 CNC  | 2026-11-06 19:49:15 | 2026-11-06 19:57:10
ROLLBACK;
```

5. Evolução do Sistema

5.1 Próximas Evoluções

Funcionalidades previstas para as próximas versões do LumiFlow:

- Concluir o relatório de tempos de produção, hoje apenas um protótipo visual, com apontamento de tempos e paradas de máquina;
- Emitir relatórios em PDF e planilha (produção por período, refugo por motivo, consumo de vidro);
- Criar indicadores de eficiência, como OEE e taxa de refugo por setor e por máquina;
- Permitir o lançamento pelo celular com leitura de QR Code da ordem no chão de fábrica;
- Ampliar o controle de estoque para chapas metálicas, componentes elétricos e demais insumos;
- Integrar o sistema a ERPs e enviar notificações de atraso e de estoque mínimo.

Referências

BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. UML: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2012.

ELMASRI, R.; NAVATHE, S. B. Sistemas de banco de dados. 7. ed. São Paulo: Pearson, 2018.

FLYWAY. Flyway documentation. Disponível em: https://documentation.red-gate.com/flyway. Acesso em: 6 nov. 2026.

OBJECT MANAGEMENT GROUP. Business Process Model and Notation (BPMN), version 2.0.2. 2014. Disponível em: https://www.omg.org/spec/BPMN/2.0.2. Acesso em: 6 nov. 2026.

ORACLE. The Java language specification: Java SE 21 edition. Disponível em:

https://docs.oracle.com/javase/specs/. Acesso em: 6 nov. 2026.

OWASP FOUNDATION. OWASP Top Ten. Disponível em: https://owasp.org/www-project-top-ten/. Acesso em: 6 nov. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 18 documentation. Disponível em: https://www.postgresql.org/docs/18/. Acesso em: 6 nov. 2026.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

SOMMERVILLE, I. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

SPRING. Spring Boot reference documentation. Disponível em: https://docs.spring.io/spring-boot/. Acesso em: 6 nov. 2026.

SPRING. Spring Security reference. Disponível em: https://docs.spring.io/spring-security/reference/. Acesso em: 6 nov. 2026.

THYMELEAF. Tutorial: using Thymeleaf. Disponível em: https://www.thymeleaf.org/documentation.html. Acesso em: 6 nov. 2026.